

1 Routing Problem Topologies and Formulations

The domain of combinatorial optimization encompasses a vast hierarchy of routing paradigms. These problems govern the spatial and temporal traversal of graphs to satisfy varying operational requirements, ranging from pure distance minimization to complex multi-period inventory synchronization, arc servicing, and facility location. The problems below are organized by the structural feature that distinguishes each family — whether the decision unit is a node, an arc, or a mixed topology — and are progressively refined by layering real-world constraints (capacity, time windows, stochasticity, profits) onto the base formulation.

1.1 The Traveling Salesperson Problem (TSP) Family

The foundational baseline for all vehicle routing. A single uncapacitated agent navigates a fully connected graph to visit a set of nodes subject to varying structural constraints.

1.1.1 Symmetric TSP (STSP) ✓

The canonical routing problem. Given a complete undirected graph $G = (V, E)$ where $c_{ij} = c_{ji}$, the objective is to find the minimum-cost Hamiltonian cycle. Using binary edge variables $x_{ij} \in \{0, 1\}$, the ILP minimizes $\sum_{i \neq j} c_{ij} x_{ij}$ subject to degree constraints ($\sum_j x_{ij} = 2$ for all i). Subtour elimination is enforced via Dantzig-Fulkerson-Johnson (DFJ) constraints for every proper subset $S \subset V$:

$$\sum_{i \in S, j \notin S} x_{ij} \geq 1$$

1.1.2 Asymmetric TSP (ATSP) ✓

A directed generalization where $c_{ij} \neq c_{ji}$, modelling one-way streets, elevation asymmetry, or time-dependent travel. Defined on a directed graph $G = (V, A)$. The LP relaxation is tightened by the Assignment Problem bound rather than spanning tree bounds, and subtour elimination requires directed cut constraints. Frequently solved via transformation to STSP using a gadget that replaces each node with a pair.

1.1.3 Generalized TSP (GTSP) ×

Partitions the node set into m disjoint clusters $C_1, \dots, C_m$. The vehicle must visit exactly one node from each cluster, minimizing the total inter-cluster tour cost. GTSP generalizes TSP ($m = |V|$, singleton clusters) and provides a natural model for location selection combined with routing. It is typically solved by transformation to an equivalent ATSP.

1.1.4 Prize-Collecting TSP (PCTSP) ✓

A selective visitation model. Each node i carries a profit p_i and a penalty γ_i if omitted. The vehicle must collect at least a minimum total prize $P_{\min}$. The objective minimizes travel cost plus penalties for unvisited nodes:

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij} + \sum_{i \in V} \gamma_i (1 - y_i) \quad \text{s.t.} \quad \sum_{i \in V} p_i y_i \geq P_{\min}$$

where $y_i \in \{0, 1\}$ activates node i .

1.1.5 TSP with Time Windows (TSPTW) ×

Injects strict service schedules. Each node i must be serviced within a hard time window $[e_i, l_i]$. Arrival time t_i at node i satisfies:

$$t_i + s_i + c_{ij} \leq t_j + M(1 - x_{ij}), \quad e_i \leq t_i \leq l_i$$

where s_i is the service duration. Soft time windows replace hard bounds with penalty terms, allowing violations at a cost.

1.1.6 Clustered TSP (CTSP) ×

Requires the vehicle to complete a full visit to every node within each cluster before departing to the next cluster. Unlike GTSP, all nodes are mandatory. The CTSP decomposes into two sequential decisions: cluster visitation order (an inter-cluster TSP) and intra-cluster sequencing (an independent TSP per cluster).

1.2 The Vehicle Routing Problem (VRP) Family

The VRP extends the TSP to a fleet of vehicles dispatching from one or more depots to service a distributed customer set. The family spans deterministic to stochastic, static to dynamic, and cost-minimizing to profit-maximizing formulations.

1.2.1 Capacitated VRP (CVRP) ✓

The fundamental multi-vehicle routing problem. A homogeneous fleet of K vehicles, each with capacity Q , services customers V_c with deterministic demands q_i . Capacity is enforced via fractional cut inequalities: for any subset $S \subseteq V_c$, the minimum number of vehicles $\lceil \sum_{i \in S} q_i / Q \rceil$ must cross the cut:

$$\sum_{i \in S, j \notin S} x_{ij} \geq 2 \left\lceil \frac{\sum_{i \in S} q_i}{Q} \right\rceil$$

1.2.2 VRP with Time Windows (VRPTW) ×

Integrates customer-specific time windows $[e_i, l_i]$ into the CVRP. Each vehicle simultaneously tracks elapsed time and remaining capacity as consumed resources along its route, exponentially increasing state-space complexity. The standard exact approach is Branch-and-Price, where the pricing subproblem is an Elementary Shortest Path Problem with Resource Constraints (ESPPRC) tracking time and load jointly.

1.2.3 Open VRP (OVRP) ×

A variant where vehicles are not required to return to the depot after completing their route. This models scenarios where drivers finish their shifts at the last customer site or vehicles are stationed regionally. Mathematically, depot-return arcs are removed from the feasible arc set, and the route cost accounts only for the outbound path. The OVRP is structurally equivalent to a CVRP in which the return arc costs are zeroed out, but optimal solutions differ substantially in topology.

1.2.4 Multi-Depot VRP (MDVRP) ×

Extends the CVRP to D geographically distinct depots, each with its own fleet and potentially distinct vehicle types. Each customer must be assigned to exactly one depot, and the routes from that depot must remain feasible under its fleet capacity. The assignment-routing coupling makes

MDVRP significantly harder than CVRP: let $z_{id} \in \{0, 1\}$ be the assignment of customer i to depot d , then:

$$\sum_{d \in D} z_{id} = 1 \quad \forall i \in V_c, \quad \sum_{i \in V_c} q_i z_{id} \leq Q_d \quad \forall d \in D$$

1.2.5 VRP with Backhauls (VRPB) $\times$

Partitions customers into linehaul nodes L (requiring outbound delivery from depot) and backhaul nodes B (requiring inbound pickup to depot). A classical precedence constraint mandates that all linehaul customers on a route are serviced before any backhaul customer, reflecting vehicle loading constraints. The mixed variant (VRPMB) relaxes this ordering. Capacity tracking requires monitoring the net vehicle load as it decreases through linehaul stops and re-increases through backhaul stops.

1.2.6 Vehicle Routing Problem with Profits (VRPP) $\checkmark$

A selective routing paradigm that bridges the CVRP and the TOP. Like the TOP, a fleet of vehicles collects profits from selectively visited nodes to maximize total network revenue. However, instead of being constrained by individual route distance or time budgets ($T_{\max}$), the vehicles are constrained strictly by **physical load capacity** Q . A vehicle may visit as many nodes as possible, provided the sum of the node demands q_i collected on the route does not exceed its physical capacity:

$$\max \sum_{i \in V} p_i y_i \quad \text{s.t.} \quad \sum_{i \in V} q_i y_{ik} \leq Q \quad \forall k, \quad \sum_k y_{ik} \leq 1 \quad \forall i$$

1.2.7 Split Delivery VRP (SDVRP) $\times$

Relaxes the single-vehicle-per-customer constraint of classical CVRP. A customer demand $q_i > Q$ may be split across multiple vehicles, or splitting may be economically optimal even when $q_i \leq Q$. Flow variables w_{ik} represent quantities delivered by vehicle k :

$$\sum_{k=1}^K w_{ik} = q_i \quad \forall i \in V_c, \quad \sum_{i \in R_k} w_{ik} \leq Q \quad \forall k$$

1.2.8 Heterogeneous Fleet VRP (HFVRP) $\times$

Models a realistic fleet with varying vehicle types, each with distinct capacity Q_k , fixed activation cost f_k , and variable arc cost c_{ij}^k . The objective balances fixed fleet deployment costs against variable routing costs, creating a coupled vehicle assignment and routing problem.

1.2.9 Periodic VRP (PVRP) $\times$

A multi-period generalization over horizon H . Each customer i requires service frequency f_i times per period and selects from a set of allowable visit-day patterns P_i . Let $y_{ip} \in \{0, 1\}$ activate pattern p for customer i :

$$\sum_{p \in P_i} y_{ip} = 1 \quad \forall i, \quad z_{it} = \sum_{p \in P_i} a_{pt} y_{ip}$$

where z_{it} drives the daily capacitated CVRP subproblem on day t .

1.2.10 Multi-Period VRP with Profits (MPVRPP) ✓

Combines the PVRP with profit-based selective visitation over horizon H . Nodes accumulate profit dynamically — either through time-varying values p_{it} or physical fill-level accumulation. The solver jointly decides *which* nodes to visit and *when*, maximizing cumulative profit subject to daily fleet budgets and capacity:

$$\max \sum_{t \in H} \sum_{i \in V} p_{it} y_{it}$$

This paradigm is the direct routing counterpart of the Inventory Routing Problem and relies heavily on mandatory selection heuristics and stochastic look-ahead.

1.2.11 Roll-on/Roll-off VRP (RRVRP) ×

A specialized variant for heavy waste logistics. Vehicles (tractors) transport single large containers (skips); a tractor carries exactly one container at a time. The state space shifts from volumetric tracking to discrete tractor states:

$$S \in \{\text{Empty}, \text{Carrying Empty Container}, \text{Carrying Full Container}\}$$

Routing consists of complex depot-customer-disposal sequences, and node degree constraints are relaxed to permit multiple visits to both customer sites and disposal facilities within a single tour.

1.2.12 Green VRP / Electric VRP (GVRP / EVRP) ×

Extends the CVRP with energy consumption constraints. In the EVRP, each vehicle has a battery capacity E that depletes along arcs proportionally to distance and load, and must be replenished at recharging stations $F \subset V$. Decision variables track both routing x_{ij}^k and battery state e_{ik} at each node. The non-linear energy depletion function — often depending on vehicle speed, gradient, and payload — is typically linearized or approximated. The GVRP uses fuel or emissions budgets in place of electric charge.

1.3 The Orienteering Problem (OP) Family

This family models selective routing scenarios where customer visitation is optional, and the goal is to maximize economic profit. The constraints dictating how many nodes a vehicle can visit define the specific problem variant.

1.3.1 Orienteering Problem (OP) ✓

A selective routing paradigm for a single vehicle where nodes carry economic profits and visitation is optional. The vehicle operates under a strict temporal or distance budget $T_{\max}$. The objective pivots from cost minimization to profit maximization:

$$\max \sum_{i \in V} p_i y_i \quad \text{s.t.} \quad \sum_{(i,j) \in A} c_{ij} x_{ij} \leq T_{\max}, \quad y_i \leq \sum_j x_{ji}$$

1.3.2 Team Orienteering Problem (TOP) ×

The multi-vehicle extension of the standard Orienteering Problem. A fleet of K vehicles, each constrained by an individual temporal or distance budget $T_{\max}^k$, collectively maximizes total collected profit. Each node may be visited at most once across the entire fleet:

$$\max \sum_{i \in V} p_i y_i \quad \text{s.t.} \quad \sum_{(i,j) \in A} c_{ij} x_{ij}^k \leq T_{\max}^k \quad \forall k, \quad \sum_k y_{ik} \leq 1 \quad \forall i$$

1.3.3 Thief Orienteering Problem (ThOP) ✓

A multi-component variant combining the Orienteering Problem with the 0-1 Knapsack Problem. A single agent (the thief) navigates a graph to steal items located at various customer cities and must return to the depot within a strict time budget. Unlike the standard OP, the vehicle's travel speed is not constant; it degrades linearly as the accumulated knapsack weight w increases:

$$v(w) = v_{\max} - \frac{w}{W}(v_{\max} - v_{\min})$$

where W is the maximum knapsack capacity, $v_{\max}$ is the empty travel speed, and $v_{\min}$ is the fully-loaded travel speed. Consequently, the travel time between cities i and j becomes state-dependent: $t(i, j) = d(i, j)/v(w_{\text{current}})$. The objective is to maximize the total profit of collected items without exceeding the dynamic time budget or the weight capacity constraints.

1.4 Stochastic and Dynamic VRP Extensions

These variants inject uncertainty or real-time dynamism into the deterministic VRP framework. They are particularly important for multi-period operational routing where demand is observed only gradually.

1.4.1 Stochastic VRP (SVRP) ✓

Demands q_i and/or customer presence y_i are random variables revealed only upon vehicle arrival. The canonical model is the VRP with Stochastic Demands (VRPSD): if a customer's realized demand exceeds remaining vehicle capacity upon arrival, the vehicle must make a detour back to the depot (a *failure*). The objective minimizes expected total routing cost including the expected failure penalty:

$$\min \sum_{(i,j)} c_{ij} x_{ij} + \mathbb{E}[\text{Failure Cost}(x, \tilde{q})]$$

Solved via stochastic programming, sample average approximation (SAA), or dynamic programming over demand distributions.

1.4.2 Dynamic VRP (DVRP) ×

Customer requests arrive online during the execution horizon, rather than being known in advance. The solver must respond in real time, re-routing the active fleet to incorporate new requests while honoring the service commitments already made. The degree of dynamism is quantified as the fraction of requests that arrive after dispatch. DVRP is typically modelled as a rolling-horizon MDP, with re-optimization triggered at fixed intervals or upon significant new request clusters.

1.4.3 Robust VRP (RVRP) ×

A worst-case formulation that optimizes against an adversarial uncertainty set $\mathcal{U}$ for demands or travel times, rather than an assumed distribution. The minimax formulation guarantees feasibility for every realization within $\mathcal{U}$:

$$\min_x \max_{q \in \mathcal{U}} \{ \text{Cost}(x, q) \mid \text{Constraints}(x, q) \}$$

Ellipsoidal and box uncertainty sets yield tractable second-order cone or robust linear programs; budget uncertainty sets (Bertsimas-Sim) yield linear reformulations directly.

1.5 The Pickup and Delivery Problem (PDP) Family

A synchronization variant where freight originates and terminates at arbitrary customer nodes rather than at a central depot.

1.5.1 Standard PDP ✓

Each request is a paired pickup–delivery tuple (i^+, i^-) . Precedence constraints enforce $t_{i^+} \leq t_{i^-}$ (pickup before delivery), and coupling constraints require the same vehicle k to service both endpoints:

$$\sum_j x_{i^+j}^k = \sum_j x_{i^-j}^k \quad \forall k$$

The vehicle load L_{ik} fluctuates continuously as pickups add weight and deliveries remove it, requiring dynamic tracking along the route.

1.5.2 Dial-a-Ride Problem (DARP) ×

The human-transport variant of the PDP (paratransit, on-demand ride-sharing). In addition to paired precedence and time window constraints, DARP enforces passenger ride-time limits L_i bounding the maximum in-vehicle duration:

$$t_{i^-} - t_{i^+} \leq L_i$$

This severely restricts the feasible solution space by coupling the spatial sequencing with individual passenger service quality, forcing meta-heuristics to balance fleet utilization against inconvenience penalties.

1.5.3 VRP with Simultaneous Pickups and Deliveries (VRPSPD) ×

Unlike standard PDP, nodes are not paired. Each customer i simultaneously requires an outbound delivery d_i from the depot and supplies an inbound pickup p_i destined for the depot. The vehicle capacity Q must be respected at all points in the tour; the exact payload on edge (i, j) must satisfy $w_{ij} \leq Q$ accounting for the cumulative net load from deliveries and pickups up to that arc.

1.6 The Arc Routing Problem (ARP) Family

Problems where operational demand is distributed along network edges rather than at discrete nodes — for example, street sweeping, snow plowing, and utility inspection.

1.6.1 Chinese Postman Problem (CPP) ×

Given a graph $G = (V, E)$, find the minimum-cost closed walk traversing every edge at least once. If the graph is Eulerian (all nodes have even degree), the optimal tour cost equals the sum of all edge weights. If not, a minimum-weight perfect matching over odd-degree nodes determines which edges require deadheading (traversal without demand).

1.6.2 Rural Postman Problem (RPP) ×

Generalizes the CPP to a required-edge subset $E_R \subset E$. The vehicle must traverse all $e \in E_R$ at least once and may traverse non-required edges to maintain connectivity. Let $x_{ij} \geq 0$ count deadheading traversals; the objective minimizes total traversal cost subject to flow conservation:

$$\min \sum_{(i,j) \in E} c_{ij} x_{ij} + \sum_{(i,j) \in E_R} c_{ij}, \quad \sum_j (x_{ij} - x_{ji}) = 0 \quad \forall i$$

1.6.3 Capacitated Arc Routing Problem (CARP) ×

Extends the RPP with a homogeneous fleet of capacity Q . Each required edge $e \in E_R$ carries a demand $q_e > 0$. The total demand of edges serviced by a single vehicle may not exceed Q . Vehicles start and end at a central depot. Binary variables y_e^k activate service of edge e by vehicle k ; integer variables x_{ij}^k track deadheading flow.

1.6.4 Periodic CARP (PCARP) ×

A multi-period extension of the CARP mirroring the PVRP structure. Required arcs carry service frequencies, and the fleet is scheduled across horizon H to satisfy all arc demands while respecting daily vehicle capacities and allowable service patterns.

1.7 The General Routing Problem (GRP) Family

A unifying framework that absorbs both node routing (VRP/TSP) and arc routing (ARP/CARP) into a single formulation.

1.7.1 Standard GRP ×

Given graph $G = (V, E)$, a required-node subset $V_R \subseteq V$ must be visited and a required-edge subset $E_R \subseteq E$ must be traversed in a single minimum-cost closed walk. The GRP is universal:

- $V_R = V, E_R = \emptyset \Rightarrow \text{TSP}$
- $V_R = \emptyset, E_R = E \Rightarrow \text{CPP}$
- $V_R = \emptyset, E_R \subset E \Rightarrow \text{RPP}$

Subtour elimination constraints must be dynamically formulated to ensure the walk covers all of $V_R \cup E_R$ as a single connected component.

1.7.2 Capacitated GRP (CGRP) ×

Injects fleet capacity Q into the GRP. Both required nodes $v \in V_R$ and required edges $e \in E_R$ may carry payload demands. Vehicles must route through the mixed topology without breaching capacity at any point.

1.8 The Location Routing Problem (LRP) Family

Couples strategic facility-siting decisions with operational vehicle routing, optimizing both simultaneously rather than sequentially.

1.8.1 Standard LRP ×

Given candidate depot locations D and customers V_c . Binary variable $z_j \in \{0, 1\}$ activates depot j at fixed cost F_j . The objective harmonizes capital expenditure with routing cost:

$$\min \sum_{j \in D} F_j z_j + \sum_k \sum_{(i,j)} c_{ij} x_{ij}^k$$

Routes must originate only from open depots: $\sum_i x_{ji}^k \leq z_j \forall j \in D, \forall k$.

1.8.2 Capacitated LRP (CLRP) ×

Adds depot throughput limits W_j to the LRP. The total demand assigned to depot j may not exceed its capacity:

$$\sum_{i \in V_c} q_i y_{ij} \leq W_j z_j \quad \forall j \in D$$

where $y_{ij} \in \{0, 1\}$ assigns customer i to depot j .

1.8.3 Multi-Echelon LRP / Two-Echelon VRP (ME-LRP / 2E-VRP) ×

Models a tiered urban distribution network. A primary fleet transports freight from large depots to intermediate satellite facilities (1st echelon). A secondary fleet of smaller vehicles distributes from satellites to end customers (2nd echelon). The formulation simultaneously optimizes satellite locations, 1st-echelon synchronization schedules, and 2nd-echelon customer routing, subject to flow conservation across both echelons and time-coupling constraints that ensure satellite inventory is replenished before downstream routes depart.

1.9 The Inventory Routing Problem (IRP) Family

The IRP transcends pure spatial routing by integrating the tactical tier of Vendor-Managed Inventory (VMI). It optimizes vehicle routes, delivery quantities, and node-level inventory states jointly over a multi-period temporal horizon $\mathcal{T}$.

1.9.1 Standard IRP ✓

A deterministic multi-period integration. For node i at period t : inventory I_{it} , unit holding cost h_i , delivered quantity q_{it} , deterministic consumption d_{it} . The objective minimizes routing plus inventory holding costs:

$$\min \sum_{t \in \mathcal{T}} \left(\sum_{(i,j) \in A} c_{ij} x_{ijt} + \sum_{i \in V} h_i I_{it} \right)$$

Governed by inventory flow conservation:

$$I_{it} = I_{i,t-1} + q_{it} - d_{it}, \quad 0 \leq I_{it} \leq C_i \quad \forall t \in \mathcal{T}$$

1.9.2 Stochastic IRP (SIRP) ✓

Consumption d_{it} is a random variable. To prevent stockouts, SIRP uses robust buffers or chance constraints bounding the stockout probability:

$$\Pr[I_{i,t-1} + q_{it} - d_{it} < 0] \leq \alpha$$

This replaces deterministic inventory bounds with statistical confidence intervals derived from the demand distribution variance.

1.9.3 Multi-Commodity IRP (MCIRP) ×

Models simultaneous distribution of distinct product families P . Delivery variables gain a commodity index q_{it}^p , requiring exact tracking of sub-compartments within vehicle capacity and distinct per-commodity depletion rates at each node.

1.9.4 Robust IRP (RIRP) ×

A worst-case variant combining robust optimization with inventory routing. Demand d_{it} is drawn from an uncertainty set $\mathcal{U}$, and the solution must be feasible for all realizations. The minimax formulation guarantees zero-stockout service under the worst-case demand trajectory within the budget set, typically yielding a tractable MILP reformulation via the Bertsimas-Sim approach.

1.9.5 IRP with Lateral Transshipment (IRP-LT) ×

Extends the standard IRP by permitting direct inventory transfer between customer nodes (lateral transshipment), rather than routing all replenishment exclusively through the depot. This is modelled by introducing arc variables f_{ijt} tracking the quantity shipped from node i to node j at period t , subject to balance constraints and vehicle capacity on the transshipment arcs. IRP-LT is particularly effective in settings where customers have complementary demand fluctuations, enabling one node to buffer another without depot intervention.

Taxonomy of Routing Problem Families and Variants

Table 1: Routing problem classifications and their distinguishing structural constraints.

Family	Key Variants	Distinguishing Constraint / Feature
TSP	STSP, ATSP	Uncapacitated, single agent, full node coverage (symmetric or directed).
	PCTSP, TSPTW	Selective visitation with penalties (Prize-Collecting) or hard time windows.
VRP	CVRP, VRPTW	Homogeneous fleet, strict vehicle capacities, and time limits.
	MDVRP, HFVRP	Multiple geographical depots (MDVRP) or heterogeneous vehicle capacities/costs (HFVRP).
	VRPP, TOP	Selective routing maximizing collected profit under strict distance/time budgets.
PDP	PVRP, MPVRPP	Multi-period scheduling with visit-frequency patterns and dynamic accumulation.
	DARP, VRPSPD	Paired pickup-delivery with passenger ride-time limits (DARP) or simultaneous node-level operations.
ARP/GRP	CARP, CGRP	Operational demand is distributed continuously along network edges rather than discrete nodes.
IRP	SIRP, MCIRP	Jointly optimizes routing and vendor-managed inventory with stochastic demands or multiple commodities.

Routing Problems: Formulation Selection Guide

Table 2: Recommended routing problem formulations based on operational requirements and constraints.

Operational Requirement / Constraint	Recommended Formulation
Single-Vehicle & Uncapacitated	
Basic node coverage with symmetric costs	STSP (Symmetric TSP)
Directional costs (e.g., one-way streets, elevation)	ATSP (Asymmetric TSP)
Location selection coupled with routing (pick one per group)	GTSP (Generalized TSP)
Selective visitation to hit a minimum revenue target	PCTSP (Prize-Collecting TSP)
Multi-Vehicle Fleet (Capacitated)	
Standard homogeneous fleet delivery	CVRP (Capacitated VRP)
Strict customer service schedules	VRPTW (VRP with Time Windows)
Drivers do not return to the depot after the last stop	OVRP (Open VRP)
Multiple dispatch locations requiring customer assignment	MDVRP (Multi-Depot VRP)
Varying vehicle sizes, activation costs, and capacities	HFVRP (Heterogeneous Fleet VRP)
Customer demand exceeds a single vehicle’s capacity	SDVRP (Split Delivery VRP)
Tractors transporting single large containers/skips	RRVRP (Roll-on/Roll-off VRP)
Economic & Multi-Period	
Maximizing collected revenue under a strict time/distance budget	VRPP / TOP (Team Orienteering)
Multi-day scheduling with required visit frequencies	PVRP (Periodic VRP)
Jointly optimizing delivery quantities and vehicle routes	IRP (Inventory Routing Problem)
Specialized Topologies	
Demand is located on the streets/edges (e.g., snow plowing)	CARP (Capacitated Arc Routing)
Freight originates and terminates at customer nodes	PDP (Pickup and Delivery Problem)
Human transit with strict maximum ride-time limits	DARP (Dial-a-Ride Problem)
Simultaneous drop-off and pickup at the exact same stop	VRPSPD (VRP with Sim. Pickup/Delivery)
Optimizing where to build depots alongside routing vehicles	LRP (Location Routing Problem)
Urban distribution via intermediate satellite hubs	ME-LRP / 2E-VRP (Two-Echelon VRP)
Uncertainty & Dynamism	
Demands are unknown until arrival; requires re-course detours	SVRP (Stochastic VRP)
Adversarial/worst-case demand bounding to guarantee zero stockouts	RVRP / RIRP (Robust VRP / IRP)
New customer requests arrive continuously during execution	DVRP (Dynamic VRP)

2 Neural Combinatorial Optimization for Vehicle Routing

The intersection of machine learning and operations research has produced **Neural Combinatorial Optimization (NCO)** — a family of methods that learn to extract spatial, temporal, and economic features from problem instances to predict near-optimal decisions, bypassing the need for hand-crafted heuristics or computationally expensive exact solvers.

Models in this domain are classified along two axes. By *construction strategy*: **Constructive** models build solutions from scratch (autoregressively or non-autoregressively); **Improvement** models learn to perturb and refine existing feasible solutions; **Hybrid** models parameterize classical meta-heuristics such as ALNS or ACO. By *generality*: **Instance-specific** models are trained and applied to a fixed problem class; **Multi-task** and **generalist** models handle heterogeneous variant families (CVRP, VRPTW, IRP, etc.) from a single set of weights.

2.1 1. Autoregressive Constructive Models

These models build a solution token by token using an encoder-decoder architecture. The encoder embeds the full instance graph once; the decoder attends over it at each step to select the next node.

2.1.1 Pointer Network (Ptr-Net) ✓

Vinyals et al., 2015

The foundational architecture that introduced attention as a trainable "pointer" over variable-length input sequences, resolving the fixed-output-vocabulary limitation of standard seq2seq models.

Mechanism. The encoder (LSTM) produces a hidden state e_j for each node j . At decoding step i , the decoder hidden state d_i is used to compute an unnormalized alignment score over all input positions, from which a categorical distribution over nodes is sampled or greedily decoded.

$$u_j^i = v^\top \tanh(W_1 e_j + W_2 d_i), \quad p(C_i = j \mid C_{<i}, \mathcal{P}) = \text{softmax}(u^i)_j$$

Ptr-Net was trained with supervised learning on exact solutions; later work replaced this with REINFORCE to remove the dependence on a solver oracle.

2.1.2 Attention Model (AM) ✓

Kool et al., 2018

The canonical Transformer-based routing model, which replaced the RNN encoder/decoder of Ptr-Net with Multi-Head Attention (MHA) and scaled to instances with hundreds of nodes.

Mechanism. Node embeddings are computed via a stack of MHA + feedforward layers. The decoder maintains a context vector $h_{(c)}$ — a concatenation of the graph embedding, the last visited node, and the first node (for CVRP, also the remaining capacity) — and produces a pointing distribution over unvisited nodes via a single-head attention with a clipping nonlinearity ($\tanh$ scaled by 10).

Training. REINFORCE with a rollout baseline: the baseline $b(s)$ is the cost of the greedy rollout of the best model from the previous epoch, providing a strong, low-variance signal.

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{p_\theta(\pi|s)} \left[(L(\pi) - b(s)) \nabla_\theta \log p_\theta(\pi \mid s) \right]$$

2.1.3 POMO (Policy Optimization with Multiple Optima) ✓

Kwon et al., 2020

One of the most impactful constructive models, achieving near-LKH3 quality on CVRP-100 at inference time. POMO exploits the fact that every routing problem has N equivalent optimal

starting nodes (the N -fold symmetry of the tour), using all of them simultaneously during both training and inference.

Mechanism. N rollouts are launched in parallel from each of the N starting nodes of an instance. During training, the multi-start reward mean is used as a self-supervised, instance-specific baseline, dramatically reducing gradient variance without requiring a separate baseline network. During inference, the best of the N rollouts is returned.

$$\nabla_{\theta} \mathcal{J}(\theta) = \frac{1}{N} \sum_{n=1}^N (L(\pi_n) - \bar{L}) \nabla_{\theta} \log p_{\theta}(\pi_n), \quad \bar{L} = \frac{1}{N} \sum_{n=1}^N L(\pi_n)$$

POMO is often used as the backbone for improvement models (e.g., EAS, SGBS) because its multi-start structure maps naturally onto population-based search.

2.1.4 SymNCO (Symmetry-Aware NCO) ✓

Kim et al., 2022

Generalizes the symmetry exploitation of POMO to arbitrary problem-level invariances (rotation, reflection, node permutation) via a contrastive self-supervised auxiliary loss.

Mechanism. For each instance, M semantically equivalent augmented views are generated (e.g., random 8-fold dihedral rotations of the coordinate plane). The policy is required to produce invariant embeddings across views, and the shared mean cost $\bar{L}$ across views serves as the REINFORCE baseline.

$$\nabla_{\theta} \mathcal{J}(\theta) = \frac{1}{M} \sum_{m=1}^M (L(\pi_m) - \bar{L}) \nabla_{\theta} \log p_{\theta}(\pi_m | s_m)$$

The augmentation-based baseline is strictly tighter than a separate critic network because it conditions on the exact instance topology.

2.1.5 MatNet ✓

Kwon et al., 2021

Designed for asymmetric combinatorial problems — the Asymmetric TSP, heterogeneous fleet VRP, or any setting where the transition cost $c_{ij} \neq c_{ji}$.

Mechanism. Rather than encoding nodes in isolation and querying pairwise costs implicitly, MatNet encodes the full cost matrix $C \in \mathbb{R}^{N \times N}$ via a bilinear cross-attention that reads the directed edge cost c_{ij} directly into the attention logit.

$$a_{ij} = \frac{\exp(q_i^{\top} k_j + w \cdot c_{ij})}{\sum_l \exp(q_i^{\top} k_l + w \cdot c_{il})}$$

This makes MatNet the preferred architecture for Fleet-Mix VRP variants, where different vehicle types create an inherently asymmetric cost structure.

2.1.6 LEHD (Light Encoder, Heavy Decoder) ×

Luo et al., 2023

Addresses the inference bottleneck of standard constructive models: while AM and POMO use symmetric encoder/decoder capacity, LEHD reconfigures the budget to a shallow encoder and a deep, autoregressive decoder that re-reads instance features at every decoding step.

Mechanism. Node embeddings are recomputed at each decoding step conditioned on the current partial solution state (visited set, remaining capacity, elapsed time). This replaces the fixed context vector with a dynamic, step-aware graph representation, substantially improving solution quality on VRPTW and CVRP at longer horizons without increasing parameter count.

2.1.7 Multi-Decoder Attention Model (MDAM) ✓

Xin et al., 2021

Addresses the tendency of greedy decoding to collapse to a single local optimum by learning multiple diverse routing policies simultaneously within one model.

Mechanism. K independent decoder heads, each initialized with a different learned embedding bias, are trained jointly with a diversity regularization term that penalizes overlap between the tours they produce. At inference, all K decoders run in parallel and the lowest-cost tour is returned, acting as a parallelized learned beam search without exponential branching.

2.2 2. Non-Autoregressive Constructive Models

Non-autoregressive (NAR) models predict the full solution structure in a single forward pass, treating routing as edge classification or heatmap generation over the instance graph. They are orders of magnitude faster at inference but typically require a post-processing search step to extract a feasible tour.

2.2.1 NARGNN ✓

Fu et al., 2021

Frames TSP/VRP as an edge-existence classification problem over the complete instance graph.

Mechanism. A message-passing GNN produces node embeddings h_i ; these are projected pairwise to produce a probability P_{ij} that edge (i, j) belongs to the optimal tour.

$$P_{ij} = \text{Sigmoid}(\text{MLP}([h_i \parallel h_j \parallel e_{ij}]))$$

Training uses Binary Cross-Entropy against solutions from an exact solver (Concorde, LKH3). The resulting edge heatmap is decoded into a tour via Monte Carlo Tree Search or greedy edge assembly with feasibility repair.

2.2.2 DPDP (Dynamic Programming with Deep Learning) ×

Kool et al., 2022

Combines the global optimality guarantees of dynamic programming with learned heuristics to prune the DP state space.

Mechanism. A GNN scores subsets of states in the DP table, and only the top- k highest-scoring partial solutions are retained at each DP stage (beam-DP). The GNN is trained to assign high scores to states that lie on the path to an optimal solution, effectively learning which partial tours are worth expanding.

This is particularly well-suited to problems with hard temporal constraints (VRPTW, multi-period IRP) where the DP state already encodes time-window feasibility.

2.2.3 BQ-NCO (Bi-directional Q-learning NCO) ×

Drakulic et al., 2023

Recasts solution construction as value-function estimation rather than policy learning, enabling principled exploration of the solution space without requiring rollouts.

Mechanism. A transformer encodes both the "left" partial tour (already assigned nodes) and the "right" partial tour (end-anchor constraints) simultaneously. A Q-value head scores each candidate next node based on the expected solution quality of completing the tour from that intermediate state. Bi-directionality anchors the search from both the start and end of the sequence, significantly reducing the compounding error of purely left-to-right autoregressive construction.

2.2.4 PolyNet ✓

A constructive model trained to learn multiple structurally distinct policies simultaneously, deliberately covering disjoint regions of the solution space. Rather than diversity-by-randomness, PolyNet conditions each decoder on a learned "policy token" that specifies which exploration mode to use — functioning as a neural ensemble that spans the solution landscape in a structured, learnable way.

2.3 3. Improvement and Local Search Models

Instead of building a solution from scratch, improvement models learn to iteratively perturb a feasible solution toward lower cost. They are often competitive with or superior to constructive models at large instance scales because they can exploit the structure of a good initial solution.

2.3.1 L2I (Learning to Improve) ×

Chen & Tian, 2019

One of the first end-to-end neural improvement models for VRP. L2I frames the search as a Markov Decision Process where the state is the current routing solution and the action is a local operator (2-opt, Or-opt, node relocation).

Mechanism. A two-phase encoder computes both node-level embeddings (spatial features) and route-level embeddings (sequence features). A pointer-based selector chooses which operator to apply and to which portion of the solution, trained via REINFORCE. Crucially, L2I maintains a feasibility mask that prevents the agent from selecting infeasible exchanges, making it directly applicable to CVRP and VRPTW without post-hoc repair.

2.3.2 N2S (Neural Neighborhood Search) ✓

Chen et al., 2022

A graph-encoder-based improvement model that generalizes local search by learning a continuous scoring function over all candidate node-swap actions.

Mechanism. The current routing state is encoded as a dynamic graph. N2S outputs a score matrix over all (i, j) node-pair exchanges, from which it selects the highest-scoring feasible swap. Unlike taboo search with a fixed neighborhood size, the learned scorer implicitly adapts the effective neighborhood radius to the local solution geometry.

2.3.3 DACT (Dual-Aspect Collaborative Transformer) ✓

Ma et al., 2021

Improvement model that explicitly decouples the two information streams that matter for routing refinement: what the nodes *are* (coordinates, demands) and where they *currently sit* in the solution sequence.

Mechanism. Two parallel transformer stacks — a Node-Aspect Transformer and a Positional-Aspect Transformer — process spatial and sequential features respectively. A cross-attention collaboration module fuses both streams before the action head selects a node-pair relocation. This prevents the common failure mode where improvement models forget the current tour structure when attending to spatial features.

2.3.4 NeuOpt (Neural Optimizer for k -Opt) ✓

Luo et al., 2023

A neural parameterization of k -opt that avoids the $\mathcal{O}(N^k)$ exhaustive search by directly predicting which k edges to exchange.

Mechanism. A self-attention network autoregressively selects a sequence of edges to break and reconnect, conditioned on all previously selected edges. The joint probability factorizes over the exchange tuple:

$$p(E_{\text{swap}} \mid G, S_t) = \prod_{i=1}^k p(e_i \mid G, S_t, e_1, \dots, e_{i-1})$$

NeuOpt effectively acts as a learned Lin-Kernighan heuristic, capable of discovering long-range improving chains that standard 2-opt or 3-opt miss entirely.

2.3.5 SGBS (Simulation-Guided Beam Search) ×

Choo et al., 2022

A search-time enhancement for any trained constructive policy (POMO, AM) that uses Monte Carlo simulation to evaluate beam candidates rather than relying on greedy completion.

Mechanism. At each beam expansion step, candidate partial solutions are evaluated not by their immediate log-probability but by the expected total cost under T stochastic rollouts from that partial state. This is lookahead-in-beam-search, and it converts a standard constructive model into a vastly more powerful inference-time planner without any retraining.

2.4 4. Meta-Heuristic Augmented Models

These architectures embed neural networks inside classical operations research algorithms, replacing hand-crafted heuristic components with learned counterparts while preserving the algorithmic skeleton that provides feasibility guarantees.

2.4.1 DeepACO (Deep Ant Colony Optimization) ✓

Liu et al., 2023

Replaces the static heuristic desirability measure $\eta_{ij} = 1/d_{ij}$ of classical ACO with a learned graph neural network, enabling the ants to follow instance-adaptive, constraint-aware guidance.

Mechanism. A GNN processes the instance graph and outputs a dense heuristic matrix. The transition probability for ant k at node i selecting node j becomes:

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha \cdot [\text{MLP}_\theta(h_i, h_j, e_{ij})]^\beta}{\sum_{l \in \mathcal{N}_i^k} [\tau_{il}]^\alpha \cdot [\text{MLP}_\theta(h_i, h_l, e_{il})]^\beta}$$

where τ_{ij} is the pheromone trail (updated classically) and θ is trained via REINFORCE on the ACO colony’s collective reward. DeepACO significantly outperforms classical ACO on CVRP, OP, and PDP benchmarks.

2.4.2 GFlowNet Ant Colony System ✓

Combines the emerging paradigm of Generative Flow Networks with ACO to overcome the mode-collapse failure of standard RL-trained routing policies.

Mechanism. Classical policy gradient methods collapse onto the single best tour they find during training. GFlowNets instead learn to sample solutions *proportionally to their reward* — a much more diverse distribution. The pheromone matrix is interpreted as a learned flow distribution, and training minimizes the Trajectory Balance loss:

$$\mathcal{L}(\theta) = \left(\log \frac{Z_\theta \cdot P_F(\tau)}{R(x) \cdot P_B(\tau)} \right)^2$$

where P_F is the forward construction policy, P_B is a fixed backward policy, $R(x)$ is solution quality, and Z_θ is a learned partition function. The result is a colony that reliably generates a diverse portfolio of high-quality solutions rather than concentrating on a single local optimum.

2.4.3 DR-ALNS (Deep RL-based ALNS) ✓

Classical ALNS selects destroy/repair operator pairs via a roulette-wheel mechanism weighted by recent performance. DR-ALNS replaces this with a DQN or PPO agent that learns the complex, instance-dependent dynamics of which operator pairs are most effective at each stage of the search.

Mechanism. The state s_t encodes the current optimality gap, search iteration count, recent operator performance history, and fleet utilization metrics. The action $a_t \in \Omega_{\text{destroy}} \times \Omega_{\text{repair}}$ selects a pair from the operator catalog. The neural network learns that, for example, *relatedness removal + greedy repair* works well early in search, while *worst removal + SISR repair* is more effective near convergence — dynamics that static roulette wheels cannot capture.

2.4.4 GLOP (Global-Local Optimization Policy) ✓

Ye et al., 2024

A hierarchical hybrid that orchestrates the handoff between a neural constructor and a classical local search solver, addressing the well-known weakness of neural models at very large scales ($N > 500$).

Mechanism. A high-level RL policy partitions the instance into subproblems (spatial clustering), assigns each subproblem to a neural constructive model for an initial solution, then hands the partial state to a fast classical solver (LKH3) for local refinement. The partition policy is trained end-to-end on the combined neural + classical objective, learning to create subproblems that are optimally sized for the downstream solver.

2.5 5. Predict-Once Solvers

Predict-once solvers decouple the neural model from the search entirely. The DNN runs a single forward pass over the instance and outputs supplementary information — edge probability heatmaps, candidate adjacency lists, cost adjustments, or branching priors — that is handed to an independent OR solver or search algorithm. The solver then operates in its own loop without any further neural inference. This clean separation means the OR solver retains its feasibility guarantees while benefiting from learned, instance-aware guidance.

The key distinction from non-autoregressive constructive models (Section 2) is that NAR models output a representation that *is* the solution (decoded directly into a tour), whereas predict-once models output guidance that an entirely separate solver consumes as an input signal.

2.5.1 GCN Edge Heatmap + Beam Search ✓

Joshi et al., 2019

The canonical predict-once architecture. A Graph Convolutional Network is trained (via supervised learning against exact solutions) to output, for every edge (i, j) in the instance graph, the probability that the edge belongs to the optimal tour. This heatmap is computed once, then a classical beam search assembles a feasible tour by following the highest-probability edges subject to routing constraints.

Mechanism. The GCN produces node embeddings h_i via L message-passing layers. Edge probabilities are computed from pairwise projections:

$$P_{ij} = \text{Sigmoid}(\text{MLP}([h_i \parallel h_j \parallel e_{ij}]))$$

Beam search then decodes the heatmap by expanding partial tours in order of cumulative edge probability, pruning branches that violate capacity or time-window constraints. Because the GCN runs only once, inference is fast and scales to instances far beyond what autoregressive models can decode greedily.

This architecture and its variants serve as the backbone of most subsequent predict-once work; NARGNN (Section 2) is a direct descendant.

2.5.2 DIFUSCO (Diffusion-based Combinatorial Optimization) ×

Sun & Yang, 2023

Applies denoising diffusion probabilistic models (DDPMs) — the same family behind image generation — to combinatorial optimization by framing the solution heatmap as a structured image to be denoised.

Mechanism. The training process diffuses a binary edge-solution matrix $x_0 \in \{0, 1\}^{N \times N}$ (the ground-truth tour) forward with Gaussian noise over T timesteps to produce $x_T \approx \mathcal{N}(0, I)$. A graph transformer learns the reverse process, predicting x_0 from x_t conditioned on the instance graph G :

$$p_\theta(x_{t-1} \mid x_t, G) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t, G), \Sigma_\theta(x_t, t, G))$$

At inference, the model runs the full reverse diffusion chain (predict-once across all T denoising steps), producing an edge probability heatmap that is decoded via greedy tour assembly or MCTS. DIFUSCO outperforms GCN-based heatmap models on TSP-500 and TSP-1000 without any fine-tuning, because the iterative denoising naturally integrates global tour consistency across the generation process.

The full reverse-diffusion chain is still a single "prediction episode" in the predict-once sense — the OR solver (MCTS, greedy) receives the completed heatmap and runs independently.

2.5.3 Neural Candidate List Generation for LKH ✓

Multiple works (*Hottung et al.; Fu et al.*) train a GNN to produce a sparse candidate adjacency list — a small set of promising successor edges for each node — that replaces LKH3’s internal greedy-distance heuristic for candidate generation.

Mechanism. LKH3 normally considers the k -nearest neighbors (by Euclidean distance) as candidate next moves. A GNN trained on historical optimal tours replaces this with a *learned* k -candidate list per node, where candidates are ranked by predicted edge inclusion probability. LKH3 then runs its full local search unchanged, but now explores a richer, constraint-aware neighborhood rather than a purely geometric one.

The quality of LKH3’s solution on CVRP and VRPTW improves by 1–3% at equal iteration budgets, because the learned candidates steer the k -opt chains away from clearly suboptimal edges early in the search.

2.5.4 ML-guided Branch and Bound ×

Khalil et al., 2016; Gasse et al., 2019

A family of methods that learn to predict branching variable scores or node selection priorities from historical B&B traces, then fix those predictions at the start of solving so the B&B tree runs without any further neural inference.

Mechanism. The predictor (a GNN operating on the bipartite constraint-variable graph of the LP relaxation) is evaluated once at the root node or at fixed tree depths, outputting a ranking over candidate branching variables. The B&B solver uses this ranking as its variable-selection policy for the remainder of the solve, replacing the default strong-branching or pseudo-cost rules.

For routing MIPs (column generation, set-partitioning formulations of VRP), this approach reduces the number of B&B nodes explored by up to an order of magnitude on instances drawn from the same distribution as the training set.

2.6 6. Predict-Many Solvers

Predict-many solvers follow the same predict-then-search philosophy as predict-once, but the neural model is called *repeatedly* during the search — each call consuming the current search state and outputting updated guidance that restricts or redirects the solver’s next steps. This tighter

coupling between the neural predictor and the search loop enables progressive narrowing of the solution space at the cost of additional inference overhead.

The trade-off is explicit: predict-once solvers amortize neural cost to a single call (fast, less adaptive); predict-many solvers pay per search step (slower, but the guidance stays synchronized with where the search currently is).

2.6.1 DPDP (Dynamic Programming with Deep Learning) ×

Kool et al., 2022

Already introduced in Section 2 as a non-autoregressive model, DPDP is more precisely a predict-many solver: the GNN is queried at every stage of the beam-DP expansion to score candidate partial solutions, and only the top- k are retained. Each DP stage triggers a fresh GNN forward pass conditioned on the current set of surviving partial solutions.

Mechanism. At DP stage t , the surviving beam $\mathcal{B}_t$ (*the top- k partial tours from the previous stage*) is passed to the GNN, which scores every possible extension (s, j) — partial tour s extended by node j . Only the top- k extensions across all (s, j) pairs survive to form $\mathcal{B}_{t+1}$. This is repeated until all nodes are assigned.

The GNN’s predict-many loop means it sees progressively more complete solutions at each stage, allowing later calls to exploit information about which routes were pruned — information that a predict-once heatmap computed at the empty-tour stage cannot have.

2.6.2 Neural MCTS (Monte Carlo Tree Search with Neural Guidance) ×

Analogous to AlphaGo’s combination of a policy network and value network with MCTS, applied to routing. The neural network is not invoked once before search; it is queried at every MCTS node expansion and rollout.

Mechanism. Two networks — a *policy network* $p_\theta(a | s)$ that scores candidate next nodes to expand, and a *value network* $v_\phi(s)$ that estimates the expected tour cost from partial solution state s — guide the Upper Confidence Bound (UCT) criterion:

$$\text{UCT}(s, a) = Q(s, a) + c \cdot p_\theta(a | s) \cdot \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

where $Q(s, a)$ is the empirical mean tour cost from simulations through action a , and $N(s)$, $N(s, a)$ are visit counts. Every tree node expansion calls p_θ ; every leaf rollout calls v_ϕ . The repeated neural calls allow the search to concentrate on the most promising subtrees far more precisely than random rollouts, at the cost of one neural forward pass per MCTS iteration.

Neural MCTS achieves near-optimal results on TSP-100 when given sufficient iteration budgets, and scales naturally to VRP by enforcing capacity feasibility as a hard constraint on legal actions at each state.

2.6.3 Iterative Graph Sparsification ×

A class of methods that progressively prune the instance graph — removing edges the model deems unpromising — over multiple rounds of prediction, feeding each pruned graph back into the neural model for the next round.

Mechanism. Starting from the complete graph $G^{(0)}$, the GNN predicts edge inclusion probabilities $P^{(0)}_{ij}$. Edges below a threshold ϵ are pruned to produce $G^{(1)}$. The GNN is re-run on $G^{(1)}$ — now a much sparser graph with a qualitatively different topology — to produce $P^{(1)}_{ij}$, and another round of pruning follows. After R rounds, a classical solver (LKH, beam search) runs on $G^{(R)}$, whose edge count may be $\mathcal{O}(N)$ rather than $\mathcal{O}(N^2)$.

The key insight is that re-running the GNN on the pruned graph is not redundant: the changed neighborhood structure alters message-passing aggregations, so the model effectively sees a different problem geometry at each round and can correct early over-pruning errors. This approach scales to TSP-10000 and VRP-5000, well beyond the range of standard constructive models.

2.6.4 Confidence-based Candidate Elimination ×

Related to iterative narrowing and curriculum-guided search

A predict-many strategy specifically for heatmap-decoded models: the neural model is re-queried on progressively smaller candidate sets, eliminating edges whose predicted inclusion probability has not exceeded a confidence threshold across multiple independent rollouts.

Mechanism. M stochastic forward passes produce M independent heatmaps. Edges where fewer than $\tau \cdot M$ passes assign probability above ϵ are eliminated from the candidate graph. The model is then re-queried on the reduced graph — now with a lower-entropy input distribution — to produce sharper edge scores for the remaining candidates. Iteration continues until the candidate graph is sparse enough for exact or near-exact decoding.

This approach is particularly effective for CVRP variants with hard constraints (time windows, precedence) where early false-positive edges in the heatmap corrupt feasibility; the multi-pass filtering catches these before the decoder commits to them.

2.7 7. Active Search and Fine-Tuning Methods

These methods improve solution quality for a *specific test instance* by updating model parameters at inference time — bridging the gap between amortized heuristics and instance-level exact solvers.

2.7.1 EAS (Efficient Active Search) ✓

Hottung et al., 2022

Active Search (Bello et al., 2016) fine-tunes the full model on each test instance via policy gradients, but this is prohibitively slow. EAS achieves comparable quality at a fraction of the cost by only fine-tuning a small set of instance-specific embedding parameters (an "adapter") while keeping the backbone frozen.

Mechanism. A lightweight adapter layer is injected into the pretrained AM or POMO model. At test time, only the adapter weights are optimized via REINFORCE on the specific instance. Because the adapter has far fewer parameters than the full model, gradient steps are cheap and convergence is fast — typically 20–50× faster than full Active Search.

2.7.2 COMPASS (Combination of Parallel Search Strategies) ×

Luo et al., 2023

A search-time strategy that runs multiple constructive policies in parallel — each initialized from a different symmetry augmentation and/or temperature — and applies local improvement operators asynchronously to the best solutions across the population. COMPASS is model-agnostic and can wrap any pretrained NCO backbone.

2.8 8. Multi-Task and Generalist Models

A significant research direction since 2023 is training a *single* model that generalizes across VRP variant families without instance-type specification at inference time.

2.8.1 MVMoE (Multi-Task VRP with Mixture of Experts) ✓

Jiao et al., 2024

Handles 16+ VRP variants (CVRP, VRPTW, OVRP, VRPB, mixed combinations) from a single set of weights using a sparse Mixture-of-Experts routing mechanism.

Mechanism. The encoder backbone is shared. A gating network routes each input token to one of E expert feedforward modules, where each expert specializes in a different constraint topology (time windows, open routes, backhauls, etc.). The gating is learned end-to-end, and

at inference, variant identity is not provided explicitly — the model infers it from the instance features. MVMoE achieves competitive performance with single-task models across all 16 variants while using roughly the same parameter budget as a single-task AM.

2.8.2 UniFIED / Omni-VRP Style Architectures ×

A class of models that encode VRP constraints as natural language or structured token sequences appended to the instance graph, enabling zero-shot generalization to novel constraint combinations not seen during training.

Mechanism. The instance (node coordinates + demands) is encoded spatially; constraint specifications (e.g., "time windows active, fleet heterogeneous, open routes") are encoded as additional tokens and cross-attended with the spatial embedding. This effectively makes the model a *conditional* routing policy, conditioned on the constraint vocabulary, rather than a fixed-variant policy.

2.9 9. Inventory Routing and Multi-Period Specific Models

The IRP adds a temporal dimension — decisions made today affect feasibility and cost tomorrow — that pure routing architectures do not natively handle.

2.9.1 L2D (Learning to Dispatch) ×

A reinforcement learning model for IRP that decouples the two interdependent decision spaces: *when and how much to deliver* (dispatch policy) vs. *in what order to visit customers* (routing policy).

Mechanism. The RL agent observes current inventory levels, demand forecasts, and vehicle positions, and outputs a dispatch decision (which customers to serve and what quantity to deliver this period). The geometric routing sub-problem is solved independently by a fast classical TSP heuristic or NCO model. Decoupling avoids the curse of dimensionality that arises from jointly optimizing route sequences and inventory levels over T periods.

2.9.2 BGN (Bipartite Graph Network) ×

Designed for the IRP and Location-Routing Problem, where the fundamental structure is a bipartite interaction between supply nodes (depots, warehouses) and demand nodes (customers).

Mechanism. Separate embedding layers encode supply-side and demand-side features. Cross-attention layers then compute a soft assignment matrix between depots and customers, which is used to initialize the routing policy. This explicit supply-demand separation avoids the representational bias of homogeneous GNNs, which treat all nodes identically.

2.9.3 Hierarchical Temporal Attention Networks ×

A family of architectures for multi-period routing that uses a two-level attention hierarchy: a *period-level* transformer attends over the T -period planning horizon to decide visit schedules, while a *within-period* transformer handles the geometric routing for a single day given the schedule.

Mechanism. Period-level attention captures inter-day inventory dynamics (the replenishment wave propagating forward through time). Within-period attention captures spatial routing efficiency. The two levels are trained jointly with a shared reward signal — total horizon cost — so the schedule level learns to create daily workloads that the routing level can execute efficiently.

This architecture is the natural neural counterpart to the Progressive Hedging / Benders decomposition approaches in the classical SIRP literature, and it is the most directly applicable NCO family to multi-period VRP.

Taxonomy of Neural Combinatorial Optimization Architectures

Table 3: Classification of NCO models by construction strategy and mechanism.

Category	Key Architectures	Core Mechanism
Autoregressive Constructive	Ptr-Net, AM, POMO, SymNCO	Builds solutions token-by-token; decoder attends over the graph embedding at each step.
Non-Autoregressive	NARGNN, DPDP	Outputs an edge-probability heatmap in a single pass; requires separate decoding (e.g., MCTS, greedy).
Improvement / Local Search	L2I, N2S, NeuOpt, DACT	Learns to iteratively select and apply topological perturbations (e.g., k -opt) to existing feasible routes.
Meta-Heuristic Augmented	DeepACO, DR-ALNS, GLOP	Replaces hand-crafted heuristic components in classical OR algorithms with trained neural networks.
Predict-Once	GCN + Beam Search, DIFUSCO	Single forward pass yields guidance (heatmaps, candidate lists) that an independent classical solver consumes.
Predict-Many	DPDP, Neural MCTS	Repeated neural queries within a search loop to dynamically score and guide tree expansion.
Multi-Task / Generalist	MVMoE, Omni-VRP	A single set of weights handles multiple routing variants via sparse routing or conditional constraint tokens.

Summary: Model Selection Guide

Table 4: NCO model selection guide.

Setting	Recommended architectures
Single-period CVRP, $N \leq 200$	POMO, AM + EAS, DACT
Single-period CVRP, $N > 500$	GLOP, Iterative Graph Sparsification, SGBS
Asymmetric costs / heterogeneous fleet	MatNet, MVMoE
Hard time windows (VRPTW)	LEHD, DPDP, L2I
Multi-variant generalization	MVMoE, Omni-VRP
Inventory routing / multi-period	L2D, BGN, Hierarchical Temporal Attention
Classical meta-heuristic augmentation	DeepACO, DR-ALNS, GFlowNet ACS
Instance-level fine-tuning at test time	EAS, COMPASS, SGBS
Single neural pass + classical solver	GCN Heatmap + Beam Search, DIFUSCO, Neural Candidate Lists for LKH
Progressive search-space restriction	DPDP, Neural MCTS, Iterative Graph Sparsification

Category trade-off summary

Table 5: Category trade-off summary for NCO models.

Category	Solution quality	Inference speed	Scalability (N)
Autoregressive constructive	Medium-High	Fast	Medium (≤ 500)
Non-autoregressive constructive	Medium	Very fast	High
Improvement / local search	High	Medium	High
Meta-heuristic augmented	High	Medium-Slow	High
Predict-once	Medium-High	Fast (1 neural call)	Very high
Predict-many	Very high	Slow (many neural calls)	High
Active search / fine-tuning	Very high	Slow	Medium
Multi-task generalist	Medium	Fast	Medium

3 Mandatory Node Selection Strategies

Domain-specific filtering algorithms that determine which nodes *must* be obligatorily visited on a given operational day in multi-period combinatorial optimization (such as the Vehicle Routing Problem with Profits and the Inventory Routing Problem). These strategies serve two distinct roles: pure **overflow prevention** (guaranteeing no node exceeds its physical capacity) and **economic prioritization** (selecting the most profitable subset given routing constraints). Some strategies operate purely in the temporal or fill-level domain; others exploit spatial routing structure to select nodes that yield maximum density or synergy.

3.1 Static Baselines

3.1.1 Regular Selection ✓

A static, periodic scheduling policy operating entirely in the time domain, independent of stochastic accumulation states. Collection is triggered for all eligible nodes on operational day t if:

$$t \equiv 1 \pmod{X}$$

for a fixed frequency X (e.g., every 3 days). Computationally trivial but highly inefficient in stochastic environments; serves as a worst-case baseline for fixed-route comparisons.

3.1.2 K-Means Geographic Sector Selection ✓

A static, spatially-aware cyclic scheduling policy that pre-partitions the node set into G geographic sectors via K -means clustering on bin coordinates, then mandates exactly one sector per operational day on a round-robin rotation. On day t the active sector index is:

$$\sigma(t) = (t - 1) \bmod G$$

and all nodes assigned to sector $\sigma(t)$ are obligatorily collected. The clustering minimises within-sector spatial variance, ensuring co-collected nodes are geographically proximate and naturally compressing per-day routing distance relative to a random or index-based partition. This formalises the dominant real-world practice of dividing service areas into fixed geographic zones assigned to specific weekdays, making it the most operationally realistic static baseline. Unlike Regular Selection (which mandates all nodes simultaneously every X days), the sector rotation guarantees a bounded, predictable daily workload of approximately n/G nodes. Spatially blind to fill-level dynamics and incapable of reacting to stochastic accumulation states; serves as the primary benchmark for strategies exploiting fill information or routing synergy.

3.1.3 Staggered Regular Selection ✓

A static, temporally distributed variant of Regular Selection that eliminates the all-or-nothing load spikes of the base policy by assigning each node i (0-indexed) a fixed phase offset:

$$\phi_i = i \bmod X$$

where X is the collection period. Node i is mandated on operational day t if and only if:

$$(t - 1) \bmod X = \phi_i$$

The uniform distribution of offsets across $\{0, \dots, X - 1\}$ guarantees that on every day exactly $\lfloor n/X \rfloor$ or $\lceil n/X \rceil$ nodes are scheduled, converting the global period X into n independent per-node schedules that collectively cover the horizon without overlap. Crucially, phase assignment is derived from array position rather than spatial coordinates, making this strategy computationally free and parameter-identical to Regular Selection (sharing the same X via `threshold`) while producing a fundamentally different load profile. Serves as the canonical load-balanced temporal baseline, isolating the cost of ignoring fill-level state from the cost of ignoring spatial structure.

3.1.4 Bernoulli Trial Random Selection ✓

A stochastic null baseline in which each eligible node i is independently mandated via a Bernoulli trial with fixed probability $p \in [0, 1]$:

$$X_i \sim \text{Bernoulli}(p), \quad i = 1, \dots, n$$

The realised selection set $S = \{i : X_i = 1\}$ has cardinality distributed as $|S| \sim \text{Binomial}(n, p)$, with $\mathbb{E}[|S|] = np$ and $\text{Var}[|S|] = np(1 - p)$. Unlike fixed- K random selection, the Bernoulli formulation decouples individual bin decisions: no bin's inclusion affects any other's probability, faithfully modelling a memoryless per-node assessment. This variability in set size is intentional — it reflects the stochasticity inherent in ignoring all state information entirely. Any deterministic strategy that fails to significantly outperform Bernoulli selection at matched expected cardinality ($K = np$) has a validity concern, making this the essential sanity-check baseline for the full strategy suite. The probability p is supplied via the shared `threshold` parameter and is clipped to $[0, 1]$.

3.2 Overflow Minimization and Risk Management

3.2.1 Last-Minute Selection ✓

A reactive, fill-level-driven strategy that triggers an obligatory collection when a node’s current fill ratio breaches a predefined critical threshold $\tau \in [0, 1]$:

$$\frac{c_i}{Q_{\max}} \geq \tau$$

where c_i is the current fill level and $Q_{\max}$ the maximum physical capacity. Acts as the primary overflow prevention backstop but suffers from spatial myopia — it considers no neighboring nodes and ignores routing synergy.

3.2.2 Deadline-Driven Selection ✓

A deterministic temporal strategy that projects the exact operational deadline for each node via:

$$d^* = \left\lfloor \frac{Q_{\max} - c}{\mu} \right\rfloor$$

where μ is the mean daily accumulation rate. Any node whose deadline d^* falls within the solver’s look-ahead horizon is immediately mandated, preventing guaranteed overflows while remaining fully deterministic and parameter-free.

3.2.3 Look-Ahead Selection ✓

A predictive, simulation-driven strategy that synchronizes multi-day collections. It identifies critically full nodes S_{crit} and simulates their time-to-overflow to find the nearest required service day d_{next} . All unassigned nodes $j \notin S_{\text{crit}}$ whose fill level projects to overflow before the vehicle returns are mandated:

$$c_j + \mu_j \cdot d_{\text{next}} \geq Q_{\max}$$

By propagating the scheduling horizon from the most urgent node outward, this strategy naturally clusters co-incident collections, reducing fleet deployment frequency.

3.2.4 Linear Service Level Prediction ✓

A statistical confidence-bound strategy for strict SLA adherence. It projects the worst-case future fill level over horizon D using a linear accumulation variance approximation:

$$w_{\text{future}} = w_{\text{current}} + D\mu + Dk\sigma$$

If this upper confidence bound breaches $Q_{\max}$, the node is mandated for immediate collection. The confidence factor k governs the risk tolerance: larger k triggers earlier, more conservative collections.

3.2.5 Stochastic Regret Selection ✓

A statistically rigorous strategy based on Expected Overflow Regret (EOR). Let $s_i = Q_{\max} - c_i$ be remaining capacity; assume daily accumulation $X \sim \mathcal{N}(\mu, \sigma^2)$ with standard score $Z = (s_i - \mu)/\sigma$. The closed-form expected overflow (the regret of deferral) is:

$$\mathbb{E}[\max(0, X - s_i)] = \sigma\phi(Z) + (\mu - s_i)(1 - \Phi(Z))$$

Collection is mandated if this expectation exceeds an acceptable risk parameter γ . Unlike threshold-based strategies, this formulation directly quantifies the economic loss of inaction.

3.2.6 Multi-Day Overflow Probability Selection ✓

A stochastic look-ahead strategy modeling accumulation over a K -day temporal horizon. Leveraging i.i.d. variance scaling ($\sigma_K = \sigma\sqrt{K}$), the strategy mandates collection if the tail probability of the K -day cumulative accumulation exceeding remaining capacity $Q_{\max} - c$ breaches an acceptable statistical risk threshold, naturally imposing tighter collection requirements on high-variance nodes.

3.2.7 Conditional Value-at-Risk (CVaR) Selection ✓

A risk-averse paradigm evaluating tail risk of capacity violations. Future fill level $F \sim \mathcal{N}(c + \mu, \sigma^2)$; surplus variable $X = F - Q_{\max}$. Rather than thresholding the expected overflow, the strategy mandates collection if the expected overflow in the worst $(1 - \alpha)$ fraction of outcomes — $\text{CVaR}_\alpha(\max(0, X))$ — exceeds a critical tolerance. CVaR is strictly more conservative than EOR-based selection, making it appropriate for penalty-heavy SLA regimes.

3.2.8 Wasserstein Distributionally Robust Selection ✓

A robust optimization paradigm replacing Gaussian point-estimate assumptions with a Wasserstein-1 ambiguity ball of radius ϵ centered on the empirical demand distribution. Using the duality results of Mohajerin Esfahani and Kuhn (2018), it optimizes against the worst-case distribution within the ball. For the ReLU overflow loss function, the worst-case expectation simplifies to the nominal expectation plus ϵ , yielding a tractable, distribution-free risk bound that requires no parametric assumption on the demand distribution.

3.2.9 MIP Multiple-Knapsack Selection (Overflow-Minimizing Variant) ✓

An exact 0/1 MILP formulation explicitly engineered to minimize expected overflow losses across a look-ahead scenario tree. Models the homogeneous fleet as K knapsacks of capacity Q . For each bin i and scenario s with probability π_s , let $o_i^{(s)} = \max(0, w_i^{(s)} - 100\%)$ be the projected overflow if not collected. The solver selects binary collection vector $x_i \in \{0, 1\}$ to minimize:

$$\min_x \sum_i (1 - x_i) \left[\sum_s \pi_s \cdot o_i^{(s)} \cdot \hat{m}_i + P_i \cdot \Pr[\text{any overflow}_i] \right]$$

Lexicographic count constraints ($\sum_i x_{i,k} \geq \sum_i x_{i,k+1} \ \forall k$) eliminate fleet symmetry from the branch-and-bound search space.

3.3 Profit Maximization and Economic Strategies

3.3.1 Revenue Threshold Selection ✓

A value-based heuristic prioritizing nodes by the absolute monetary value of their current contents. Let V be total volumetric capacity, ρ waste density, R_{kg} unit revenue. A node is mandated if its expected revenue exceeds a profitability threshold τ_{rev} :

$$\left(\frac{c_i}{Q_{\max}} \right) V \rho R_{\text{kg}} > \tau_{\text{rev}}$$

3.3.2 Profit-per-Kilometer (Spatial ROI) Selection ✓

A routing-aware economic heuristic that normalizes expected node value by a proxy for marginal insertion cost. The spatial Return on Investment is computed as $\text{Score}_i = r_i / (2d_{0,i})$, deprioritizing highly valuable but isolated nodes. A node is mandated only when this economic-spatial ratio exceeds a configured threshold, naturally avoiding costly detours for marginal profit.

3.3.3 Fractional Knapsack (Density-Greedy) Selection ✓

A high-speed $\frac{1}{2}$ -approximation of the profit-maximizing multiple-knapsack problem. Nodes are sorted by net-profit density (profit per unit mass) and greedily packed into vehicle capacities. The $\frac{1}{2}$ -approximation guarantee is maintained by returning the maximum between the greedy-packed set and the single most profitable node that fits independently.

3.3.4 Lagrangian Reduced-Cost Selection ✓

A relaxation-based strategy evaluating the LP relaxation of the profit-maximizing multiple-knapsack problem. The dual variable (shadow price) λ^* of the binding global capacity constraint is extracted. Nodes with strictly positive Lagrangian reduced cost are mandated:

$$\bar{c}_i = (r_i - \text{cost} \cdot d_i) - \lambda^* m_i > 0$$

This provides an economically grounded prioritization: mandated nodes are exactly those whose marginal value exceeds the capacity’s shadow price, aligning individual collection decisions with the system-level economic optimum.

3.3.5 Filter-and-Fan Selection ✓

A meta-heuristic selection strategy adapted from Glover (1998) that balances economic value with capacity constraints through a two-phase search.

1. **Filter Phase:** Evaluates all unassigned nodes using a composite score that amplifies separable net profit by spatial urgency: $\text{Score}_i = (r_i - 2d_{0,i}) \cdot C \cdot \left(1 + \frac{c_i}{Q_{\max}}\right)$. The top k scoring nodes are retained to form an initial seed set (the filter beam).
2. **Fan Phase:** Systematically explores an add/remove neighborhood around the seed set up to a maximum search depth. Because the net-profit objective is mathematically separable, marginal move evaluations are $\mathcal{O}(1)$. The algorithm greedily accepts additions that improve total profit and ejects nodes that degrade it.

A fail-safe mechanism then deterministically forces the inclusion of any node breaching the critical fill threshold τ , ensuring operational safety alongside economic optimization.

3.4 Spatial and Synergistic Routing Strategies

3.4.1 Spatial Synergy Selection ✓

A geometric, density-aware strategy that amortizes fixed travel costs by co-locating collections. Critically full nodes S_{crit} are identified via a severe threshold τ_{crit} . A moderately full node j is mandated if it lies within Euclidean radius R of any critical node:

$$S_{\text{syn}} = \left\{ j \mid \frac{c_j}{Q_{\max}} \geq \tau_{\text{syn}} \wedge \exists i \in S_{\text{crit}} : d_{i,j} \leq R \right\}$$

The final selection set is $S_{\text{crit}} \cup S_{\text{syn}}$. The bipartite threshold structure prevents trivially nearby low-fill nodes from being included.

3.4.2 Clarke-Wright Savings Selection ✓

A spatial synergy filter derived from the Clarke-Wright VRP heuristic. Pairwise spatial savings are computed as $s_{ij} = d_{0,i} + d_{0,j} - d_{i,j}$. A node is mandated only if it satisfies a minimum fill requirement *and* exhibits strictly positive savings with at least one other critically full node, suppressing isolated routing in favor of cluster formations.

3.4.3 Route-Cluster Synergy Selection $\times$

A routing-state-aware strategy that conditions node selection on the *current partial route structure* rather than purely on fill levels or pairwise distances. For each unassigned node i , the marginal insertion cost into the nearest active route R^* is computed:

$$\Delta C_i = \min_{(u,v) \in R^*} (d_{u,i} + d_{i,v} - d_{u,v})$$

A node is mandated if its fill level exceeds a moderate threshold *and* its marginal insertion cost relative to its expected revenue satisfies $\Delta C_i / r_i \leq \epsilon_{\text{route}}$. Unlike static spatial synergy (which ignores current fleet positioning), this strategy adapts dynamically to where vehicles actually are — mandating nodes that the active fleet can absorb cheaply on its current trajectory.

3.4.4 Set-Cover (Hub) Selection $\checkmark$

A geometric aggregation strategy that maps required visits to a Minimum Set Cover formulation. It defines a universe U of critically full nodes and seeks a minimum-cardinality hub set such that every node in U lies within service radius R of a chosen hub. Solved via a greedy algorithm providing a theoretically guaranteed $\ln(|U|)$ approximation ratio.

3.4.5 Submodular Facility Location Selection $\checkmark$

A subset selection strategy maximizing a submodular coverage objective with diminishing returns:

$$f(S) = \sum_{i \in \text{Bins}} \max \left(0, r_i - \alpha \min_{j \in S \cup \{0\}} d_{i,j} \right)$$

Maximized under a cardinality budget via the Lazy Greedy algorithm (Minoux, 1978), which exploits a max-priority queue of marginal gains to avoid redundant computations while maintaining a $(1 - 1/e)$ approximation guarantee.

3.4.6 Supermodular Synergy Selection $\checkmark$

A clustering strategy maximizing an objective with increasing returns (supermodularity), balancing expected revenue against a lower-bound TSP tour length approximation:

$$f(S) = \sum_{i \in S} r_i - 2\alpha \sum_{i \in S} \min_{j \in S \setminus \{i\}} d(i, j)$$

The marginal routing cost decreases as the cluster grows, enabling localized clusters to collectively cross the viability threshold. A modified greedy approach with continuous re-evaluation of non-positive candidates allows the algorithm to construct dense, jointly profitable clusters that no node could justify individually.

3.5 Simulation and Decision-Theoretic Strategies

3.5.1 One-Step Rollout (Approximate DP) Selection $\checkmark$

A simulation-based look-ahead formalizing the decision as an optimal stopping problem. For each node, the expected future reward of two actions — "Collect Today" vs. "Defer to Tomorrow" — is evaluated. The "Defer" trajectory is simulated over finite horizon H using a base heuristic (e.g., Last-Minute selection) and discounted by factor γ . A node is mandated if the expected discounted reward of immediate collection strictly dominates deferral.

3.5.2 Multi-Step Rollout Selection ✓

Extends the one-step rollout by simulating the *full* multi-day trajectory rather than a single-step comparison. For each candidate collection set S , a simulation policy is rolled out over horizon H to estimate total expected routing cost and overflow penalties:

$$V(S) = \mathbb{E} \left[\sum_{t=1}^H \gamma^t (C_{\text{route},t}(S) + C_{\text{overflow},t}(S)) \right]$$

The set $S^* = \arg \min_S V(S)$ is selected, with the expectation approximated via Monte Carlo sampling of demand trajectories. Multi-step rollout captures temporal dependencies that one-step lookahead misses — for instance, choosing to collect a node today to prevent a costly detour in three days — at the cost of higher computational overhead.

3.5.3 Whittle Index (Restless Multi-Armed Bandit) Selection ✓

A state-of-the-art reinforcement learning baseline modelling the system as a Restless Multi-Armed Bandit (RMAB). Each node is an independent MDP with two actions: active (collect) or passive (accumulate). The Whittle Index is the exact "subsidy for passivity" m at which the decision-maker is indifferent between the two actions. Computed via Value Iteration over a discretized state space, nodes are ranked by their indices and the top K are selected, naturally balancing urgency and economic value over infinite horizons.

3.5.4 Value Function Approximation (VFA) Selection ×

An approximate dynamic programming strategy that replaces the exact Value Iteration of Whittle Index computation with a parameterized function approximator $\hat{V}_\theta(s_i)$ — for instance, a linear combination of fill ratio, accumulation rate, time since last collection, and depot distance. The parameters θ are fitted offline via temporal difference learning or regression on simulated rollout data. At deployment, each node is scored by $\hat{V}_\theta$, and nodes whose VFA score exceeds a threshold are mandated. VFA trades the exact optimality guarantee of Whittle for dramatically lower per-instance computation.

3.5.5 Pareto-Front (Non-Dominated) Selection ✓

A multi-objective combinatorial strategy discarding scalar weighting in favor of strict Pareto dominance. Each node is evaluated in a two-dimensional objective space: Urgency (expected days to overflow) and Routing Efficiency (spatial distance to depot). Nodes on the non-dominated Pareto frontier are mandated, ensuring no node is excluded if it is simultaneously more urgent *and* cheaper to route than any alternative.

3.6 Adaptive and Ensemble Dispatchers

3.6.1 Combined (Ensemble) Selection ✓

A meta-selection architecture that evaluates multiple independent base strategies concurrently and aggregates their discrete selection outputs via logical operators:

$$S_{\text{final}} = \bigcup_{k \in K} S_k \quad (\text{Logical OR — aggressive union})$$

$$S_{\text{final}} = \bigcap_{k \in K} S_k \quad (\text{Logical AND — conservative intersection})$$

The OR mode is appropriate when any single strategy's signal is considered actionable; the AND mode is appropriate when unanimous agreement is required before commitment.

3.6.2 Portfolio (Ensemble) Dispatcher ✓

A macro-level dispatcher that aggregates decisions from multiple *heterogeneous* strategies (e.g., CVaR, Whittle, Savings) via the same logical OR/AND aggregation as the Combined strategy. The architectural distinction from Combined Selection is the strategy composition: Portfolio Dispatcher is designed for strategies with fundamentally different information sources (risk, spatial, economic) whose outputs are complementary, whereas Combined Selection typically aggregates strategies from the same domain using varied parameterizations.

3.6.3 Contextual Thompson Sampling Dispatcher ✓

A Multi-Armed Bandit meta-dispatcher that dynamically selects among candidate selection strategies (e.g., exact MIP vs. fast greedy) based on current environment state. A Bayesian Beta-Bernoulli posterior (α_k, β_k) is maintained for each strategy k . At each operational day, a success probability is sampled $\theta_k \sim \text{Beta}(\alpha_k, \beta_k)$ and modulated by a temperature parameter. The strategy with the highest sampled value is dispatched. Posteriors are updated multiplicatively based on the resulting routing cost improvement over the rolling baseline, continuously re-weighting toward strategies that work best on current instance geometry and fill-level distributions.

3.6.4 Entropy-Regularized Adaptive Selection ×

A meta-dispatcher that adds an explicit exploration bonus to prevent premature strategy specialization. The selection score for each strategy k at day t is:

$$\text{Score}_k = \bar{R}_k - \beta_t H(\pi_k)$$

where $\bar{R}_k$ is the empirical mean reward of strategy k , $H(\pi_k)$ is the entropy of its selection distribution over nodes (low entropy = highly deterministic, high entropy = diverse), and β_t is an annealing exploration coefficient. Strategies producing diverse, spread-out selections are preferred early in the search horizon (high β_t); concentrated, high-precision strategies are preferred as the horizon closes ($\beta_t \rightarrow 0$).

3.6.5 Learned Imitation Selection ✓

A machine-learning heuristic that bypasses the computational bottleneck of exact MILP solvers. A pre-trained classification model (Random Forest or Neural Network) extracts a localized state vector per node: fill ratio, accumulation rate, demand variance, spatial distance, net revenue. A forward pass predicts the probability that the exact solver would select the node, and nodes exceeding a threshold probability are mandated. Unlike the VFA strategy (which approximates the value function), Learned Imitation directly imitates the exact solver’s binary selection decision, making it faster but less interpretable.

Mandatory Node Selection Strategies

Table 6: Domain-specific filtering strategies for multi-period and profit-aware routing.

Category	Strategy	Primary Mechanism
Static & Reactive	Regular / Last-Minute	Fixed frequency scheduling or strict capacity thresholding ($\geq \tau$).
Risk Management	Deadline-Driven	Projects exact days until guaranteed overflow using mean accumulation μ .
	CVaR / Wasserstein	Evaluates statistical tail risk or distributionally robust ambiguity balls to prevent stock-outs under uncertainty.
Economic	Spatial ROI	Normalizes expected node revenue by its marginal routing insertion cost ($r_i/2d_{0,i}$).
	Lagrangian Reduced-Cost	Evaluates LP relaxation shadow prices; mandates nodes with strictly positive marginal utility.
Spatial Synergy	Clarke-Wright Savings	Flags nodes exhibiting minimum fill requirements and positive spatial savings ($s_{ij} > 0$) with critical nodes.
Simulation	Whittle Index	RMAB reinforcement learning formulation computing the exact subsidy for passivity via Value Iteration.
Dispatchers	Contextual Thompson	Multi-Armed Bandit dynamically selecting the optimal filtering strategy based on current environment state.

Mandatory Selection: Strategy Selection Guide

Table 7: Recommended mandatory selection strategies based on operational settings.

Setting / Constraint	Recommended Strategies
Strict SLA adherence / Penalty avoidance	Linear Service Level Prediction, Deadline-Driven
Unknown or adversarial stochastic demand	Wasserstein Distributionally Robust, CVaR Selection
Profit maximization & Economic routing	Lagrangian Reduced-Cost, Profit-per-Kilometer (ROI)
Highly clustered geographic topologies	Spatial Synergy, Clarke-Wright Savings, Hub Selection
Infinite-horizon dynamic accumulation	Whittle Index (Restless Multi-Armed Bandit)
Multi-objective/Unpredictable environments	Contextual Thompson Sampling, Portfolio Dispatcher

Mandatory Selection: Category Trade-off Summary

Table 8: Trade-offs across mandatory node selection categories.

Category	Overflow Risk	Spatial Efficiency	Computation	Look-ahead
Static & Reactive	High	Very Low	Instant	None
Risk Management	Low	Low	Medium	Statistical
Economic / Value	High	Medium	Fast	Marginal
Spatial / Synergy	Medium	Very High	Medium	Geometric
Simulation / MDP	Very Low	High	Very Slow	Multi-step
Ensembles / Dis-patchers	Very Low	High	Slow	Adaptive

4 Combinatorial Optimization Operator Taxonomy

At the core of modern combinatorial optimization — particularly in complex vehicle routing and scheduling landscapes — lies a highly orchestrated ecosystem of algorithmic operators. Rather than relying on a single monolithic solver, state-of-the-art frameworks (LNS, ALNS, memetic algorithms, hyper-heuristics) construct solutions by dynamically chaining together specialized functional units.

These operators serve five distinct strategic purposes:

1. **Construction and Repair** — Building feasible topological states from scratch (Initialization) or intelligently restoring feasibility after massive structural disruptions (Recreate).
2. **Exploitation (Descent and Intensification)** — Aggressively optimizing a localized region of the search space to reach a strict mathematical local minimum (Improvement).
3. **Exploration (Diversification and Perturbation)** — Dismantling or altering configurations to force the trajectory out of deep local optima basins (Destroy, Shaking, Mutation).
4. **Synthesis (Recombination)** — Merging the most successful structural features of multiple parent solutions into superior offspring (Crossover).
5. **Meta-Control (Orchestration)** — Governing *which* operator to apply, *when*, and in what *sequence*, often using machine learning or performance memory (Search Heuristics, Sequence Merging).

Let the routing topology be defined on graph $G = (V, E)$ with capacity bounds Q .

4.1 Crossover (Recombination)

Crossover operators synthesize a new offspring solution by inheriting structural features from two or more parent solutions. Let parents be denoted P_1 and P_2 and the offspring as O .

4.1.1 Sequence-Based Crossovers

Ordered Crossover (OX) ✓ Selects a contiguous subsequence from P_1 and copies it directly to O . The remaining nodes $V \setminus O$ are inserted in the exact order they appear in P_2 , omitting duplicates. Perfectly preserves local relative ordering and guarantees Hamiltonian feasibility without generating sub-tours. Complexity: $\mathcal{O}(|V|)$.

Partially Mapped Crossover (PMX) × Selects two random cut points defining a segment in P_1 which is copied directly to O . The remaining positions are filled from P_2 , using a position-mapping defined by the overlapping segment to resolve duplicates. PMX preserves the absolute positions of nodes in the crossover segment while maintaining a globally valid permutation, making

it particularly effective in VRP variants where position along a route is semantically meaningful (e.g., time window satisfaction).

Cycle Crossover (CX) $\times$ Identifies cycles in the permutation mapping between P_1 and P_2 . Nodes in odd cycles are inherited from P_1 ; nodes in even cycles are inherited from P_2 . CX guarantees that every node occupies a position held by the same node in at least one parent, strictly preserving absolute positional structure. This is appropriate when the identity of the node at a specific route position matters more than the relative order of nodes.

Random Node Inheritance $\checkmark$ A uniform crossover where each node v_i 's route assignment is inherited from either P_1 or P_2 via an independent Bernoulli draw $X \sim \text{Bernoulli}(0.5)$. Capacity violations are frequent and mandate a secondary $\mathcal{O}(|V| \log |V|)$ feasibility repair phase to unassign and heuristically reinsert violating nodes.

Sequential Constructive Crossover (SCX) $\times$ A routing-specific crossover that constructs the offspring by greedily selecting the next node from either P_1 or P_2 at each step. Starting from the first node, the algorithm looks ahead in both parents from the current position and chooses the edge leading to the unvisited node with the lower insertion cost. SCX tends to produce offspring with shorter total route cost than OX or PMX because the construction is guided by the objective function, not purely by inherited structure.

4.1.2 Structural and Partition Crossovers

Position-Independent Crossover (PIX) $\checkmark$ Inherits elements based on their absolute assignment to specific vehicle routes rather than sequence position. For each route $R_k \in P_1$, a corresponding route $R_j \in P_2$ is identified; the offspring inherits the intersection $R_k^O = R_k \cap R_j$. Highly effective for cluster-first, route-second architectures where route membership is more stable across generations than within-route ordering.

Selective Route Exchange Crossover (SREX) $\checkmark$ Operates at the macroscopic route level. A subset of structurally intact, high-quality routes in P_1 (evaluated by density or cost-per-demand) is identified and swapped with geographically intersecting routes in P_2 . Duplicate nodes are purged from the less optimal routes to restore bipartite graph validity.

Generalized Partition Crossover (GPX) $\checkmark$ A highly advanced operator that identifies independent sub-graphs (partitions) where the two parent tours differ. Let G_Δ be the symmetric difference of edges $E(P_1) \oplus E(P_2)$. The algorithm identifies mutually exclusive connected components within G_Δ , exhaustively evaluates each partition, and swaps the optimal sub-graph assignments between parents. The offspring is mathematically guaranteed to satisfy $f(O) \leq \min(f(P_1), f(P_2))$.

Route Profit — Generalized Partition Crossover $\checkmark$ A variant of GPX formulated for orienteering problems. Partitions are evaluated by net revenue: $\Delta\text{Profit} = \sum_{i \in S} p_i - \lambda \sum_{(i,j) \in S} d_{ij}$. Sub-graphs are swapped only if the net profit of the offspring strictly increases.

4.1.3 Edge-Based Crossovers

Edge Recombination Crossover (ERX) $\checkmark$ Focuses on adjacency preservation. An edge-map $M(v)$ of all neighbors for each node v across $E(P_1) \cup E(P_2)$ is constructed. The offspring O is built by iteratively transitioning to the available neighbor in $M(v)$ that possesses the fewest unvisited neighbors, strongly preserving inherited geometric edge structures.

Capacity-Aware Edge Recombination Crossover (CA-ERX) ✓ Extends ERX by enforcing dynamic vehicle capacity limits Q during construction. Any edge transition (u, v) that would cause route load $\sum_{i \in R} q_i + q_v > Q$ is explicitly pruned from the adjacency map, preventing infeasible offspring generation without a post-hoc repair phase.

4.1.4 Multi-Period Crossovers

Pattern and Itinerary Crossover ✓ Designed for Periodic VRPs (PVRP). The offspring inherits visit-day frequency matrices $A_{it} \in \{0, 1\}$ (temporal schedule assignments) from P_1 , while inheriting the spatial routing sequences (geometric itinerary) for those specific days from P_2 . This decoupling of temporal and spatial inheritance allows independent optimization of the visit schedule and the routing topology.

4.2 Destroy (Ruin)

Destroy operators map a feasible solution s to a partial solution s' by ejecting a subset of nodes U ($|U| = k$) into an unassigned pool. All operators feature a standard distance/cost variant and a `_profit` variant where applicable.

4.2.1 Stochastic and Cardinality-Based Removals

Random Removal ✓ The baseline exploration operator. Samples k nodes from $V \setminus \{0\}$ via a uniform PMF $P(X = v) = 1/n$. Guarantees asymptotic traversal of the full search space and prevents deterministic stalling. Complexity: $\mathcal{O}(k)$.

4.2.2 Cost and Proximity-Based Removals

Worst Removal ✓ Evaluates the marginal objective contribution $\Delta f_i = f(s) - f(s_{-i})$ of every active node. In VRPs, this is the localized detour cost: $\Delta C_i = d_{prev_i, i} + d_{i, next_i} - d_{prev_i, next_i}$. Nodes are sorted descending by ΔC_i and removed deterministically or via a randomized rank-selection parameter $p \geq 1$ to prevent cycling.

Shaw Removal ✓ Removes a cluster of related nodes to enable dense spatial reconstruction. Relatedness $R(i, j)$ is a normalized weighted sum:

$$R(i, j) = \phi \cdot \frac{d_{ij}}{\max d} + \chi \cdot \frac{|t_i - t_j|}{\max \Delta t} + \psi \cdot \frac{|q_i - q_j|}{\max \Delta q}$$

An initial seed node is selected; nodes with minimum $R(\cdot, j)$ relative to the active removed set U are iteratively ejected.

Neighbor Removal ✓ Selects seed node v_{seed} and removes its k nearest spatial neighbors regardless of current route assignments, punching a geographic hole in the solution topology.

Time-Window-Based Removal × A constraint-aware removal for VRPTW instances. Groups nodes by time window overlap: a node j is related to seed i with score $R_{tw}(i, j) = 1/(|e_i - e_j| + |l_i - l_j| + 1)$. Removing temporally clustered nodes creates large, coherent time-window gaps that the repair operator can fill more efficiently than random or spatial removal alone, because re-inserted nodes compete for the same narrow time slots.

Historical Removal ✓ Leverages a long-term search memory matrix M_{ij} tracking the frequency with which edge (i, j) appears in global-best solutions. Nodes connected by low-historical-success edges are prioritized for ejection, guiding the search away from empirically poor structures.

Penalized Removal $\times$ Uses an augmented objective $f'(s) = f(s) + \lambda \sum p_{ij}$ where p_{ij} counts consecutive iterations an edge has remained static in a local optimum. Targets and breaks the highest-penalty edges, forcing trajectory escape.

4.2.3 Macro-Structural and Geometric Removals

Cluster Removal $\checkmark$ Bypasses individual node evaluation. Executes a rapid partitioning (K-means or DBSCAN), selects centroid μ_k , and ejects all nodes assigned to that cluster, forcing macroscopic topological reconstruction.

Route Removal $\checkmark$ Ejects all nodes from a uniformly selected route R_k , forcing the repair phase to pack them into the remaining $K - 1$ fleet. Acts as a proxy for fleet-size minimization.

String Removal $\checkmark$ Removes a contiguous topological sequence $\{v_i, v_{i+1}, \dots, v_{i+L}\}$ within a single route R_k with length $L \sim U(1, L_{\max})$, preserving the overarching route framework while creating a localized, flexible gap.

Sector Removal $\checkmark$ Ejects all nodes within a geometric wedge from the depot. Given a seed angle θ_{seed} and sweep radius $\Delta\theta$, node i is ejected if $\min(|\theta_i - \theta_{\text{seed}}|, 2\pi - |\theta_i - \theta_{\text{seed}}|) \leq \Delta\theta$, cleanly severing intersecting radial routes.

Zone Removal $\times$ A concentric-ring variant of Sector Removal. Rather than an angular wedge, all nodes within an Euclidean annulus $[r_{\min}, r_{\max}]$ centered on a randomly selected node (not necessarily the depot) are ejected. This creates a circular gap in the solution topology — particularly useful for instances with ring-structured routes or depot-peripheral customer clusters.

4.2.4 Multi-Period Removals

Random Horizon Removal $\checkmark$ Flattens the T -day schedule into $(\text{node}_i, \text{day}_t)$ tuples and uniformly samples k specific visits, decoupling spatial proximity from temporal disruption.

Worst Profit Horizon Removal $\checkmark$ Evaluates temporal marginal profit $P_{i,t} = p_i - C(d_{\text{prev},i} + d_{i,\text{next}} - d_{\text{prev},\text{next}})$ for every scheduled visit. The visits with the lowest net profit are ejected, freeing temporal capacity for higher-yielding alternatives.

Shaw Horizon Removal $\checkmark$ Cross-temporal extension of Shaw. After a seed visit (i, t) is removed, subsequent removals (j, t') are sampled based on spatial proximity d_{ij} and temporal proximity $|t - t'|$, stripping spatiotemporal clusters.

Urgency-Aware Removal $\checkmark$ Identifies nodes near critical inventory threshold τ and removes their currently scheduled visits, forcing the repair phase to reschedule them earlier to avert stock-outs.

Shift Visit Removal $\checkmark$ Ejects a visit on day t with a hard re-insertion constraint: the node must be re-inserted only into day $t - 1$ or $t + 1$, sliding the schedule along the temporal axis without altering visit frequency.

Pattern Removal $\checkmark$ Erases the full visit-frequency pattern vector $A_i = [a_{i1}, \dots, a_{iT}]$ for a target node i , dumping all its scheduled instances into U for completely unconstrained temporal and spatial rescheduling.

4.3 Evolutionary Mutation

Mutation operators introduce localized, stochastic variations into offspring post-crossover to maintain population diversity and prevent premature genetic convergence.

4.3.1 Sequence and Position Mutations

Swap Mutation ✓ Selects two nodes uniformly at random and swaps their positions: $\mathcal{O}(1)$ localized perturbation.

Inversion ✓ Selects two split points p_1, p_2 and reverses the substring between them. Topologically equivalent to a randomized 2-opt edge exchange.

Scramble ✓ Selects a substring $[p_1, p_2]$ and applies a random internal permutation, destroying local sequence while preserving macro-assignments.

Random 2-Opt ✓ Identifies two non-adjacent edges (u, v) and (x, y) uniformly at random, deletes them, and reconnects as (u, x) and (v, y) . Feasibility regarding time windows is checked post-mutation.

4.3.2 Continuous-to-Discrete Mutations

Random Differential Evolution ✓ Maps the canonical DE mutation to combinatorial integer spaces. A mutant vector v_i is generated by:

$$v_i = x_{r1} \oplus F \otimes (x_{r2} \ominus x_{r3})$$

where $\oplus, \ominus, \otimes$ are mapped to heuristic edge-exchange probabilities.

Best Differential Evolution ✓ Identical to Random DE, but the base vector x_{r1} is forced to the global best individual x_{best} , aggressively biasing mutation toward exploitation.

4.4 Generalized Insertion and Deletion (GENI)

The GENI architecture bypasses the requirement that insertions and deletions occur only between currently adjacent nodes. By evaluating multiple non-adjacent reconnections simultaneously, GENI accesses sub-neighborhoods invisible to standard 2-opt or Or-opt moves.

4.4.1 Unstringing (Generalized Deletion)

Type I Unstringing ✓ Removes node V_i between adjacent V_{i-1} and V_{i+1} . Breaks four arcs and inserts three new arcs, explicitly reversing two distinct internal sub-tours:

$$\Delta C = (d_{i-1,j} + d_{i+1,k} + d_{j+1,k+1}) - (d_{i-1,i} + d_{i,i+1} + d_{j,j+1} + d_{k,k+1})$$

Type II Unstringing ✓ Involves three non-adjacent reference nodes, breaking five arcs and creating three reversed sub-tours. Complexity: $\mathcal{O}(|V|^3)$ per targeted node.

Type III Unstringing ✓ Removes V_i and reconnects using two non-adjacent reference points, reversing two sub-tours while maintaining a distinct topological orientation from Type I.

Type IV Unstringing ✓ The most computationally intensive generalized deletion ($\mathcal{O}(|V|^4)$). Uses four reference nodes, extracting two forward and two reversed segments. Restricted to heavily constrained subproblems.

4.4.2 Stringing (Generalized Insertion)

Type I Stringing ✓ Inserts candidate V_x using reference nodes V_i, V_j, V_k . Severs three arcs and establishes four new links, reversing sub-tours $(V_{i+1} \dots V_j)$ and $(V_{j+1} \dots V_k)$:

$$\Delta C = (d_{i,x} + d_{x,j} + d_{i+1,k} + d_{j+1,k+1}) - (d_{i,i+1} + d_{j,j+1} + d_{k,k+1})$$

Type II Stringing ✓ Uses two additional distant reference points (V_k, V_l) , forcing non-contiguous sub-tour reversals to test deeper insertion cavities.

Type III Stringing ✓ Explores macro-level disruption: inserts V_x while reversing nearly the entire sequence outside the immediate insertion neighborhood.

Type IV Stringing ✓ Performs a massive simultaneous insertion involving four non-adjacent reference nodes. $\mathcal{O}(|V|^4)$ complexity restricts use to highly constrained subproblems.

4.5 Improvement (Descent)

Improvement operators monotonically optimize the objective, acting as local search sinks by rejecting worsening moves ($\Delta f \geq 0$).

4.5.1 Edge-Exchange Descent

Steepest 2-Opt ✓ Exhaustively evaluates the full $\mathcal{O}(|N|^2)$ neighborhood of pairwise edge-cross removals. Caches all valid improving moves and commits strictly to the move with the maximum cost reduction $\max |\Delta C|$.

3-Opt ✓ Removes three distinct edges, fracturing the route into three sub-tours. Evaluates all 7 possible reconnecting configurations. 3-opt breaks the subset of local minima that 2-opt cannot (non-reversing configurations), at $\mathcal{O}(|N|^3)$ cost.

4-Opt Removes four edges and evaluates all valid reconnection configurations. The number of topologically distinct reconnections grows combinatorially, making exhaustive 4-opt search impractical for large routes. In practice, 4-opt is applied selectively — either via the Double-Bridge move (a specific non-sequential 4-opt configuration used in perturbation) or restricted to a small candidate set identified by the 3-opt neighborhood.

K-Opt (Variable Depth) ✓ A generalized k -edge exchange. As k increases, valid reconnection count grows exponentially; bounding heuristics (such as α -nearness candidate lists) are required to maintain computational viability.

4.5.2 Node and Sequence Descent

Steepest Or-Opt ✓ Iterates over all continuous chains of lengths $L \in \{1, 2, 3\}$. Completely extracts each chain and evaluates re-insertion at every valid position $\mathcal{O}(|N|^2)$. Commits to the maximal improvement, repeating until the neighborhood confirms a local minimum.

Steepest Node Exchange ✓ Systematically evaluates the objective delta for all pairwise node swaps across the sequence. Iterates until $\Delta f \geq 0$ for all pairs, forcing the solution strictly downhill.

4.6 Intensification (Fixing)

Operators that mathematically lock specific solution components in place, allowing exact sub-solvers to intensify within the restricted space.

4.6.1 Exact Mathematical Formulations

Exact Dynamic Programming ✓ Extracts isolated vehicle routes and formulates them as independent TSPs, solved exactly via Held-Karp DP. Viable only for $|V_{R_k}| \leq 20$ due to $\mathcal{O}(|V|^2 2^{|V|})$ complexity.

Fix and Optimize ✓ A decomposition matheuristic. A large subset of binary routing variables is locked ($x_{ij} = \bar{x}_{ij}$), and the remaining "free" variables are passed to an exact MILP solver (Gurobi, CPLEX) to find the provably optimal sub-MIP solution.

Set Partitioning Polish ✓ Aggregates all heuristically generated feasible routes into a memory universe Ω and solves an exact Set Partitioning formulation:

$$\min \sum_{r \in \Omega} c_r y_r \quad \text{s.t.} \quad \sum_{r \in \Omega} a_{ir} y_r = 1 \quad \forall i \in V, \quad y_r \in \{0, 1\}$$

4.7 Inter-Route Local Search

Operators altering macroscopic graph topology by exchanging nodes or edges between different routes, explicitly balancing vehicle capacities.

4.7.1 Segment and Suffix Exchanges

Cross Exchange ✓ Swaps a contiguous segment S_A from R_A with a segment S_B from R_B , preserving internal sequence. Capacity feasibility: $L(R_A) - \sum_{i \in S_A} q_i + \sum_{j \in S_B} q_j \leq Q$.

Lambda Interchange ✓ Exhaustively explores all valid subset exchanges between route pairs up to cardinality $\lambda_{\max}$ (Osman, 1993). Complexity: $\mathcal{O}(|R_A|^\lambda \cdot |R_B|^\lambda)$.

Improved Cross Exchange (I-CROSS) ✓ Extends Cross Exchange by evaluating four topological configurations for each segment pair: standard, S_A reversed, S_B reversed, and both reversed. Steepest descent selects the best configuration.

Or-opt* (Inter-Route Or-opt) ✓ The inter-route generalization of standard Or-opt. Chains of length $L \in \{1, 2, 3\}$ are extracted from route R_A and evaluated for re-insertion at every position in every other route $R_B \neq R_A$, subject to capacity feasibility. This is the most frequently improving inter-route operator in CVRP local search after Node Relocation, because it allows efficient transfer of coherent node chains between routes without the geometric disruption of full segment swaps.

$$\Delta C = (d_{p_u, n'_u} - d_{p_u, u_1} - d_{u_L, n_u}) + (d_{v, u_1} + d_{u_L, n_v} - d_{v, n_v})$$

where $u_1, \dots, u_L$ is the extracted chain and v, n_v is the insertion position.

4.7.2 Subramanian Neighborhoods

Shift(2,0) ✓ Extracts a contiguous block $B_u = (u_1, u_2)$ from R_A and relocates it after node v in R_B . Objective delta:

$$\Delta C = (d_{p_u, n_u} + d_{v, u_1} + d_{u_2, n_v}) - (d_{p_u, u_1} + d_{u_2, n_u} + d_{v, n_v})$$

Swap(2,1) ✓ Asymmetric exchange: block (u_1, u_2) from R_A swapped with single node v from R_B . Validates capacities on both routes: $L(R_A) - q_{u_1} - q_{u_2} + q_v \leq Q$ and $L(R_B) - q_v + q_{u_1} + q_{u_2} \leq Q$.

Swap(2,2) ✓ Symmetrical macroscopic exchange of blocks (u_1, u_2) from R_A and (v_1, v_2) from R_B . Evaluates in $\mathcal{O}(|R_A| \cdot |R_B|)$; effective for exchanging geometrically overlapping neighborhood pairs.

Cross ✓ Exchanges the exact suffixes of two routes starting after split positions $p_u \in R_A$ and $p_v \in R_B$. Severs edges $(u, u+1)$ and $(v, v+1)$, establishing crossover links. Capacity validation requires computing aggregate demand of the respective swapped tails: $\Delta Q_A = \sum_{i>u} q_i$, $\Delta Q_B = \sum_{j>v} q_j$.

4.7.3 Multi-Route and Cascading Displacements

Cyclic Transfer (p -exchange) ✓ Coordinates a simultaneous exchange of single nodes across $p \geq 3$ distinct routes, bypassing multi-route capacity deadlocks that stall simple 2-route operators. For a Forward Shift:

$$\Delta C = \sum_{i=0}^{p-1} [\Delta \text{Insert}(v_{(i-1) \bmod p}, R_i) - \Delta \text{Remove}(v_i, R_i)]$$

The sequence is committed only if strict capacity feasibility $L_i - q_{v_i} + q_{v_{i-1}} \leq Q$ is maintained across all p routes simultaneously.

Ejection Chain ✓ A recursive intensification mechanism for strict fleet-size minimization. Targets a `source_route` R_s and attempts complete unassignment, cascading displacements through the fleet: if inserting node u into route R_k violates capacity, an existing node $v \in R_k$ is displaced recursively, up to `max_depth`. Failure triggers a full localized rollback.

Exchange Chains ✓ Applies explicit cardinality bounds to sequential shifts: ejects exactly k nodes from one route while reciprocally receiving exactly h nodes, bounding the computational depth of standard ejection chains.

4.8 Intra-Route Local Search

Operators bounded strictly to mutating the topological sequence *within a single isolated route* R_k .

4.8.1 Edge-Exchange Neighborhoods

2-Opt ✓ Standard internal edge uncrossing. Removes two non-adjacent arcs and establishes the single valid Hamiltonian reconnection:

$$\Delta C = d_{i,j} + d_{i+1,j+1} - d_{i,i+1} - d_{j,j+1}$$

3-Opt ✓ Removes three distinct internal edges, fracturing the route into three sub-tours. Exhaustively evaluates all 7 reconnecting configurations, producing strictly better local minima than 2-opt at $\mathcal{O}(|N|^3)$ cost.

K-Opt ✓ A generalized k -edge exchange. The explosive reconnection count as k increases is controlled via α -nearness candidate lists.

4.8.2 Node-Displacement Neighborhoods

K-Permutation ✓ Selects k nodes within the route and evaluates all $k!$ orderings of those nodes, holding the surrounding sequence infrastructure fixed.

Relocate ✓ Removes single node u and inserts it immediately after v :

$$\Delta C = (d_{p_u, n_u} - d_{p_u, u} - d_{u, n_u}) + (d_{v, u} + d_{u, n_v} - d_{v, n_v})$$

Relocate Chain (L3) ✓ Extracts a contiguous sequence of k nodes and shifts the entire block to a new valid insertion point, checking downstream time-window violations.

Or-Opt ✓ An exhaustive first-improvement wrapper using Relocate Chain with $k \in \{1, 2, 3\}$. Repeats until $\Delta f \geq 0$.

Swap ✓ Swaps internal positions of nodes u and v . If adjacent, the objective evaluation accounts for the shared edge to prevent double-counting.

4.9 Perturbation (Shaking)

Operators designed explicitly for diversification. Their goal is to push the search trajectory completely out of entrenched local optima basins, frequently accepting massive objective degradation ($\Delta f \gg 0$).

4.9.1 Heuristic Sequence Disruptions

Double Bridge ✓ A canonical 4-opt move that severely disrupts sequence continuity. Breaks a tour into four quarters and reconnects in a non-sequential order (A-D-C-B). Critically, no standard 2-opt or 3-opt move can reverse this reconnection — it is *not* contained in the 2-opt or 3-opt neighborhoods — guaranteeing genuine trajectory escape rather than immediate regression to the previous local optimum.

Kick ✓ Dismantles a configurable fraction (e.g., 30%) of the active solution via uniform node ejection, followed by immediate greedy repair. In profit variants, destruction is biased toward nodes with poor marginal utility: $P_u = p_u - \lambda(d_{prev, u} + d_{u, next} - d_{prev, next})$.

Perturb ✓ Executes k random node swaps. A parameter `prob.unvisited` dictates the probability of swapping a routed node with a completely unassigned node $u \in U$, structurally forcing the network into entirely new geographic sectors.

4.9.2 Exact and Evolutionary Shaking

Branch and Bound Perturbation ✓ Directs an exact MILP solver to find a feasible solution lying strictly outside the current heuristic neighborhood, injecting mathematically validated but structurally novel configurations.

Evolutionary Perturbation ✓ A micro-GA triggered on a restricted spatial cluster of active routes. Flattens selected routes into a Giant Tour, applies Swap mutations, and for orienteering models, executes a "harvesting" pre-step: forcibly swapping highly profitable unvisited nodes into the sequence before re-optimization.

Genetic Transformation (GT) ✓ Compares the current state against an archived global-elite solution. Intersecting edges $E_{\text{common}} = E_{\text{current}} \cap E_{\text{elite}}$ are locked. All other nodes are ejected and stochastically reinserted, protecting historically proven macro-structures while randomizing the micro-topology.

4.9.3 Multi-Period Disruptions

Cross-Day Shuffling ✓ Perturbs multi-period schedules by shifting nodes or entire route structures across the T -day horizon, forcing evaluation of vastly different inventory and visit-frequency combinations.

4.10 Recreate (Repair)

The structural reconstruction phase. These operators process the unassigned pool U and execute topological re-insertions into the partial state s' to restore feasibility s'' .

4.10.1 Myopic and Cost-Based Insertions

Greedy Insertion ✓ Iterates over U and all valid insertion positions, greedily committing to the minimum localized cost increase $\arg \min \Delta C$. Complexity per iteration: $\mathcal{O}(|U| \cdot |V_{\text{active}}|)$.

Deep Insertion ✓ An aggressive load-balancing heuristic (Archetti et al.) penalizing fragmented vehicle space. Candidate evaluations use a composite score:

$$\text{Score} = \Delta C - \alpha \left(\frac{Q - L(r) - q_v}{Q} \right)$$

Mathematically coerces tight packing, implicitly minimizing fleet size K .

Noise-Perturbed Insertion × Adds a small stochastic noise term $\epsilon_{ij} \sim U(-\delta, \delta)$ to the insertion cost $\Delta C_{ij} + \epsilon_{ij}$ before committing. The noise breaks systematic tie-preferences between geometrically similar insertion positions, diversifying the repair output across LNS iterations without sacrificing the greedy selection structure. Particularly effective when repeated ALNS iterations produce identical repair sequences.

4.10.2 Look-Ahead and Regret Insertions

Regret Insertion ✓ Calculates the opportunity cost of deferred insertion. Let $\Delta C_{i,j}$ be the cost of inserting node i into its j -th best route. Nodes with maximum regret are prioritized:

$$\text{Regret}_i = \sum_{j=2}^k (\Delta C_{i,j} - \Delta C_{i,1})$$

Forward-Looking Insertion ✓ Expands evaluation beyond localized ΔC by incorporating a temporal penalty for downstream slack consumption. Checks the aggregate temporal slack w_i of all nodes downstream of the insertion point, penalizing insertions that restrict future flexibility.

4.10.3 Spatial and Geometric Insertions

Farthest Insertion ✓ Targets the unassigned node furthest from any currently active route sequence. Establishes extreme structural "skeletons" first, preventing tight overlapping central clusters.

Nearest Insertion ✓ Inserts the unassigned node $u \in U$ minimizing Euclidean distance to the active network V_{active} . Biases topology toward dense, short-distance loops.

4.10.4 Advanced and Exact Insertions

GENI Insertion ✓ Rebuilds tours using GENI Type I and II stringing logic, bypassing geometric adjacency requirements and evaluating deeper sub-tour reversals to weave nodes into the graph.

Greedy Blink ✓ Stochastic modification to strict greedy logic. The objectively best insertion position is deliberately ignored ("blinked") with probability p , forcing stochastic variance into the reconstruction sequence and preventing deterministic LNS cycling.

Branch and Bound Insertion ✓ Delegates reconstruction of U to an exact sub-solver, guaranteeing the absolute mathematically optimal partial insertion relative to the current route architectures.

4.11 Search Heuristics

Macro-level frameworks controlling the sequence and orchestration of low-level functional units.

4.11.1 Trajectory and Ejection Frameworks

Guided Ejection Search ✓ A high-level controller managing cascaded ejection chains, dynamically adjusting displacement depth limits based on the severity of capacity constraints.

Large Neighborhood Search ✓ The primary Destruction $\rightarrow$ Recreation orchestration loop, accepting newly synthesized states based on a pluggable Acceptance Criterion (SA, TA, GD).

4.11.2 Advanced Edge-Exchange Frameworks

Lin-Kernighan ✓ A variable-depth heuristic dynamically evaluating sequential k -opt exchanges based on continuous distance evaluations, structurally unbounding k until local optimality is proven.

Lin-Kernighan-Helsgaun (LKH) ✓ The state-of-the-art exact/heuristic hybrid. Bounds the explosive LK search space using mathematically rigorous α -nearness 1-trees, enabling extreme 5-opt and partition moves in viable computational time.

4.12 Sequence Merging

Hyper-heuristic mechanisms that mutate, construct, and optimize sequences of *other algorithmic operators*, moving the optimization layer from the physical routing graph to the algorithmic logic itself.

4.12.1 Probabilistic and Learning-Based Merging

Ant Colony Optimization Sequence ✓ Applies ACO logic to operator selection. The pheromone matrix τ_{ij} represents the historically validated probability of improving the objective by executing Operator j immediately following Operator i .

Markov Chain Sequence ✓ Maintains a row-stochastic transition matrix $T[i, j] = P(\text{Op}_j \mid \text{Op}_i)$, sampling the next operation based on the current algorithmic state.

Sequential Selection ✓ A hyper-heuristic controller tracking real-time LLH success rates and dictating the execution pipeline via RL frameworks (ϵ -greedy or Softmax).

4.12.2 Evolutionary Merging

Sequence Recombination ✓ Treats the entire executed operator sequence as a genetic chromosome. Evolves the controller logic by applying OX crossover and point mutations to the operator chains.

4.13 Solution Initialization

Foundational constructive algorithms building the initial feasible state s_0 from an empty $G = (V, E)$.

4.13.1 Greedy and Geometric Constructors

Greedy ✓ Constructs the sequence via lowest-cost ($\min d_{ij}$) or highest-profit ($\max p_i$) sequential insertion until fleet limits are met.

Nearest Neighbor ✓ Instantiates at the depot and iteratively traverses to the unvisited node minimizing d_{ij} . Computationally trivial ($\mathcal{O}(|V|^2)$) but frequently yields globally suboptimal, deeply overlapping macro-clusters.

4.13.2 Merge and Look-Ahead Constructors

Savings (Clarke-Wright) ✓ Begins with every node assigned to an isolated back-and-forth route $(0, i, 0)$. Evaluates spatial savings $s_{ij} = d_{0,i} + d_{0,j} - d_{i,j}$ and iteratively commits to the highest savings merge valid under capacity bounds Q .

Regret ✓ Applies Regret- k logic to the fully unassigned universe, prioritizing geographically isolated nodes that would incur massive penalties if deferred, building s_0 with a globally balanced initial topology.

GRASP (Greedy Randomized Adaptive Search Procedure) ✓ A hybrid constructive algorithm bridging deterministic logic with stochastic exploration. Computes greedy costs for all candidate moves, constructs a Restricted Candidate List (RCL) containing only elements within threshold α of the optimal, then samples the next move uniformly from the RCL. GRASP provides controllable randomization: $\alpha = 0$ recovers pure greedy; $\alpha = 1$ recovers pure random construction. GRASP solutions typically serve as high-quality starting points for subsequent local search, making it the standard initialization method in GRASP + Local Search meta-heuristic frameworks.

Combinatorial Search Operator Taxonomy

Table 9: Low-level combinatorial operators defining the atomic mutations of routing graphs.

Operator Class	Notable Operators	Structural Impact
Crossover	OX, PMX, GPX, ERX	Inherits absolute positions, relative sequences, or spatial partitions from two or more parent solutions.
Destroy (Ruin)	Worst, Shaw, Cluster, String	Ejects specific nodes based on marginal cost, spatial/demand relatedness, or geometric contiguousness.
Mutation	Swap, Inversion, Scramble	Introduces localized stochastic variations (e.g., reversing substrings) to maintain genetic diversity.
Generalized (GENI)	Unstringing / Stringing	Evaluates multi-arc, non-adjacent vertex insertions and deletions to bypass standard topological adjacency bounds.
Perturbation	Double Bridge, Kick	Violently fragments and reconnects quarters of a tour to ensure absolute escape from local optima basins.
Recreate (Repair)	Greedy, Deep, Farthest	Restores graph feasibility by inserting nodes to minimize cost or maximize geometric coverage.

Search Operators: Operator Selection Guide

Table 10: Recommended combinatorial operators based on targeted structural mutations.

Target Structural Mutation	Recommended Operators
Preserving exact route partitions from parents	Generalized Partition Crossover (GPX), PIX
Breaking strict time-window/spatial clusters	Time-Window Removal, Shaw Removal, Sector Removal
Bypassing strict topological adjacency limits	GENI Unstringing & Stringing (Types I-IV)
Absolute escape from 3-opt/Or-opt local minima	Double Bridge 4-opt, Kick, Random 2-Opt Mutation
Restoring topology with minimum fleet waste	Regret Insertion, Deep Insertion, Farthest Insertion

Search Operators: Category Trade-off Summary

Table 11: Trade-offs across low-level combinatorial search operators.

Operator Class	Primary Goal	Acceptance Tolerance	Complexity	Graph Impact	Im-
Crossover	Synthesis	Elitist / Generational	High	Massive	
Destroy (Ruin)	Exploration	N/A (Yields partial state)	Low-Medium	Large subset	
Evolutionary Mutation	Diversity	Generational	Low	Localized	
Improvement (Descent)	Exploitation	Strict ($\Delta f < 0$)	Medium-High	Edge / Sequence	
Perturbation (Shaking)	Escape	Always Accepted	Low	Topological	
Recreate (Re-pair)	Feasibility	N/A (Follows Destroy)	Medium	Restorative	

5 Routing Solution Algorithms

This document catalogues the full algorithmic suite available for routing optimization, spanning four distinct abstraction tiers: **Exact and Decomposition Solvers** provide provably optimal solutions with worst-case exponential cost; **Matheuristics** hybridize MILP formulations with heuristic search for strong bounded solutions; **Meta-Heuristics** guide combinatorial search trajectories through stochastic and evolutionary mechanisms; and **Hyper-Heuristics** operate above the search layer, managing and sequencing entire portfolios of lower-level algorithms.

5.1 Exact and Decomposition Solvers

Exact solvers are the gold standard for establishing provably optimal baselines. Their practical applicability is bounded by the problem’s state-space complexity, but they serve as indispensable sub-solvers and benchmarking tools within larger hybrid frameworks.

5.1.1 Branch-and-Bound (BB) ✓

The foundational search algorithm for exact MILP optimization. BB partitions the feasible decision space into smaller subproblems (branching) and computes continuous LP relaxation lower bounds $LB \leq Z^*$ at each node. If a node’s lower bound exceeds the best-known incumbent ($LB \geq UB$), or the subproblem is infeasible, the subtree is pruned. This implicitly exhausts the search space, guaranteeing global optimality without explicit enumeration.

5.1.2 Branch-and-Cut (BC) ✓

Enhances BB by embedding cutting plane generation directly into the search tree. At each node, the fractional LP solution $\tilde{x}$ is evaluated. If $\tilde{x} \notin \text{conv}(\mathcal{X})$, the algorithm generates valid inequalities (cutting planes) $\alpha^\top x \leq \beta$ separating $\tilde{x}$ from the integer polyhedron, aggressively tightening the LP relaxation and reducing the search tree exponentially.

5.1.3 Branch-and-Price (BP) ✓

Integrates Branch-and-Bound with column generation, deployed for Set Partitioning formulations with exponentially many route variables. The Restricted Master Problem (RMP) is solved on a limited column subset. A pricing subproblem — typically an ESPPRC (Elementary Shortest Path

Problem with Resource Constraints) — generates improving route columns with negative reduced cost:

$$\bar{c}_j = c_j - \sum_{i \in V} \pi_i a_{ij} - \mu < 0$$

5.1.4 Branch-and-Price-and-Cut (BPC) ✓

The state-of-the-art exact framework for routing MIPs. BPC unifies column generation and cutting planes within the B&B tree. To combat symmetry — where fractional branching yields negligible bound improvements — BPC dynamically separates advanced valid inequalities such as Subset-Row Cuts (SRCs) and lifted cover inequalities from 0-1 knapsack relaxations. The pricing subproblem is modified to incorporate dual variables from generated cuts, keeping the LP relaxation tight throughout.

5.1.5 Constraint Programming with Boolean Satisfiability (CP-SAT) ✓

Integrates finite-domain CP with modern SAT engines via lazy clause generation. Complex spatial constraints and vehicle capacities are translated dynamically into SAT clauses. When routing node i to j under capacity violation causes conflict, a nogood clause $\neg(x_i = v_1 \wedge x_j = v_2)$ is injected into the SAT solver, frequently outperforming continuous LP relaxations on highly constrained instances.

5.1.6 Integer L-Shaped Benders Decomposition (ILS-BD) ✓

An exact framework for Stochastic MILPs where second-stage recourse decisions involve discrete variables. Standard LP duality cannot generate Benders cuts for integer subproblems, so this architecture dynamically produces integer optimality cuts (Laporte & Louveaux, 1993) at integer-feasible master tree nodes. Let S^k be the index set of master variables taking value 1 at iteration k :

$$\theta \geq (Q(x^k) - L) \left(\sum_{i \in S^k} x_i - \sum_{i \notin S^k} x_i - |S^k| + 1 \right) + L$$

5.1.7 Logic-Based Benders Decomposition (LBBD) ✓

Removes the requirement that the Benders subproblem be a continuous LP. By modelling routing subproblems via Constraint Programming, LBBD derives exact bounding cuts through logical inference rather than dual rays. The master problem employs a bounding function $B_k(x)$ inferred from the CP solver’s proof of optimality or infeasibility:

$$\theta \geq B_k(x), \quad B_k(x^k) = \min_{y \in Y(x^k)} f(x^k, y)$$

5.1.8 Exact Stochastic Dynamic Programming (ESDP) ✓

Solves multi-stage stochastic routing by evaluating Bellman’s optimality principle backward through the temporal horizon. The value function $V_t(S_t)$ is computed recursively over all system states:

$$V_t(S_t) = \min_{x_t \in \mathcal{X}(S_t)} \{C(S_t, x_t) + \gamma \mathbb{E}_{\omega_t}[V_{t+1}(\mathcal{T}(S_t, x_t, \omega_t))]\}$$

Provides a theoretically perfect baseline but is bottlenecked by the curse of dimensionality in large state spaces.

5.1.9 Progressive Hedging (PH) ✓

A scenario-based decomposition that relaxes non-anticipativity constraints. Each probabilistic scenario $s \in \mathcal{S}$ is optimized independently, with an augmented Lagrangian penalty enforcing consensus toward a shared policy z :

$$\min_{x_s} \left\{ p_s f_s(x_s) + w_s^{(k)\top} x_s + \frac{\rho}{2} \|x_s - z^{(k-1)}\|^2 \right\}$$

The consensus $z^{(k)} = \sum_s p_s x_s^{(k)}$ and multipliers $w_s^{(k+1)} = w_s^{(k)} + \rho(x_s^{(k)} - z^{(k)})$ are updated iteratively until convergence.

5.1.10 Scenario Tree Extensive Form (ST-EF) ✓

The baseline approach for finite stochastic routing models. Constructs the full probabilistic scenario tree and formulates the Deterministic Equivalent Problem (DEP) as a single monolithic MILP:

$$\min_{x, y_s} \sum_{s \in \mathcal{S}} p_s (c^\top x + q_s^\top y_s) \quad \text{s.t.} \quad Ax \leq b, \quad T_s x + W_s y_s \leq h_s \quad \forall s$$

Memory footprint scales exponentially with scenario count; used primarily for small-scale stochastic benchmarks.

5.1.11 Smart Waste Collection — Two-Commodity Flow (SWC-TCF) ✓

An MILP adapted from the two-commodity network flow model (Baldacci et al., 2004). Continuous flow variables y_{ij} (waste load) and y'_{ij} (empty vehicle capacity) replace exponential subtour elimination constraints. For vehicle capacity Q and binary routing variables x_{ij} , the flow conservation is:

$$y_{ij} + y'_{ij} = Q x_{ij} \quad \forall (i, j) \in A, \quad \sum_j y_{ji} - \sum_j y_{ij} = q_i \quad \forall i$$

Maximizes $\sum_i R \cdot q_i - C \sum_{ij} d_{ij} x_{ij}$ subject to multi-period SLA thresholding.

5.2 Matheuristics

Matheuristics hybridize the speed of heuristic trajectory search with the mathematical bounding power of exact MILP formulations, inheriting the best properties of both paradigms.

5.2.1 ILS-RVND-SP (Iterated Local Search + Randomized VND + Set Partitioning) ✓

A gold-standard hybridization (Subramanian et al., 2013) operating in two tightly coupled phases.

Route Generation. ILS escapes local optima while a Randomized Variable Neighborhood Descent (RVND) performs local search. Unlike standard VND, RVND explores neighborhoods $\mathcal{N} = \{N_1, \dots, N_k\}$ in a random sequence. On any improvement, the sequence is immediately reshuffled and reset, continuing until no operator yields improvement.

Exact Selection. The diverse route pool Ω generated by ILS-RVND is passed to a Set Partitioning ILP:

$$\max \sum_{r \in \Omega} P_r x_r \quad \text{s.t.} \quad \sum_{r \in \Omega} a_{ir} x_r \leq 1 \quad \forall i \in V, \quad x_r \in \{0, 1\}$$

5.2.2 POPMUSIC ✓

A scalable decomposition architecture (Taillard & Voss, 2002). Partitions massive instances into overlapping subproblems via a KD-Tree spatial index: a seed node’s K closest route centroids are identified in $\mathcal{O}(K \log N)$ and re-optimized using a targeted local solver (ALNS or HGS). If the local solve yields cost reduction, the global solution is updated. The overlapping structure ensures boundary consistency without global re-optimization.

5.2.3 Kernel Search (KS) ✓

A restricted variable-space framework (Angelelli et al., 2010). The LP relaxation of the global MILP identifies variables with strictly positive fractional values as the initial kernel $\mathcal{K}_0$. Remaining variables are sorted by reduced cost into buckets $\mathcal{B}_1, \dots, \mathcal{B}_m$. At iteration i , a restricted MILP over $\mathcal{K}_{i-1} \cup \mathcal{B}_i$ is solved; any $\mathcal{B}_i$ variable used in the optimal solution is permanently absorbed:

$$\mathcal{K}_i = \mathcal{K}_{i-1} \cup \{j \in \mathcal{B}_i \mid x_j^* > 0\}$$

5.2.4 Adaptive Kernel Search (AKS) ✓

Extends KS (Guastaroba et al., 2017) with runtime-adaptive variable fixing. The sub-MILP solve time t_{solve} classifies each bucket as EASY, NORMAL, or HARD relative to threshold t_{easy} . For HARD instances, LP-near-optimal variables are aggressively fixed:

$$x_j = 1 \quad \forall j \in \mathcal{K} \quad \text{where} \quad \tilde{x}_j \geq 1 - \epsilon$$

For EASY instances, fixing is bypassed and the kernel expands freely to chase tighter global bounds.

5.2.5 Local Branching (LB) ✓

An exact matheuristic (Fischetti & Lodi, 2003) that restricts the MIP search space via a Hamming-distance constraint from the incumbent $\bar{x}$. Let $B_1 = \{j \mid \bar{x}_j = 1\}$ and $B_0 = \{j \mid \bar{x}_j = 0\}$:

$$\sum_{j \in B_0} x_j + \sum_{j \in B_1} (1 - x_j) \leq k$$

Restricting k prunes the tree dramatically. If the sub-MIP reaches its time limit without improvement, the neighborhood is diversified ($k \leftarrow k + 2$) or the exhausted region is permanently excluded.

5.2.6 Local Branching with VNS (LB-VNS) ✓

Embeds Local Branching within a Variable Neighborhood Search framework (Hansen et al., 2006). Three phases cycle iteratively: **Shaking** forces a starting point at the boundary of N_k (equality constraint); **Local Search** runs a tight LB solve ($k_{LS} = 4$) from the shaken point; **Neighborhood Change** resets k to $k_{\min}$ on improvement (intensification) or expands $k \leftarrow k + k_{\text{step}}$ on failure (diversification).

5.2.7 Relaxation Enforced Neighborhood Search (RENS) ✓

A primal LNS variant (Berthold, 2009) requiring no initial feasible incumbent. Solves the LP relaxation to obtain $\tilde{x}$, then fixes all naturally integer variables to their LP bounds:

$$x_j = \tilde{x}_j \quad \forall j : \tilde{x}_j \in \{0, 1\}$$

The exact solver optimizes only the remaining fractional variables, rapidly establishing a high-quality upper bound.

5.2.8 Relaxation Induced Neighborhood Search (RINS) ×

Similar to Local Branching, but fixes variables based on LP-integer agreement rather than Hamming distance from the incumbent. All variables where the continuous LP relaxation $\tilde{x}_j$ exactly matches the integer incumbent $\bar{x}_j$ are fixed; the exact solver explores only the remaining unfixed subspace, concentrating computation at the most uncertain decision boundaries.

5.2.9 Restricted Master Heuristic (Heuristic Column Generation) ×

Addresses the computational bottleneck of exact Branch-and-Price by replacing the exact pricing subproblem with a fast meta-heuristic (Tabu Search, ILS) that rapidly generates route columns with negative reduced costs. The exact MILP solver is then invoked exclusively on the Restricted Master Problem to optimally select the column combination, separating the hard combinatorial pricing from the exact selection.

5.2.10 Learning-to-Branch / Learning-Augmented Matheuristics ×

The frontier of ML-integrated matheuristics. Graph Neural Networks score and predict the outcome of strong-branching tests, filtering non-promising branches without executing the LP solver at each candidate. The search tree is traversed orders of magnitude faster while retaining mathematical rigor, because the GNN amortizes the branching cost over training instances.

5.2.11 Cluster-First Route-Second (CF-RS) ✓

A foundational two-stage paradigm (Fisher & Jaikumar, 1981). **Clustering:** the geographic space is partitioned into angular sectors with seed nodes per sector. Unassigned nodes i are allocated to seeds k via either a greedy approximation ($C_{ik} = d(0, i) + d(i, k) - d(0, k)$) or an exact Generalized Assignment Problem (GAP) MILP:

$$\max \sum_i \sum_k (R \cdot w_i - C \cdot C_{ik}) x_{ik} \quad \text{s.t.} \quad \sum_k x_{ik} \leq 1 \quad \forall i, \quad \sum_i w_i x_{ik} \leq Q \quad \forall k$$

Routing: locked clusters are solved as independent TSPs via PSO with Enhanced Edge Recombination (EER) crossover and 2-opt mutation.

5.3 Meta-Heuristics

High-level algorithmic frameworks that guide search trajectories through stochastic, memory-based, or population-driven mechanics. Organized below by primary search mechanism: trajectory-based (single-solution), evolutionary/population-based, and swarm-based.

5.3.1 Trajectory-Based Methods

Simulated Annealing (SA) ✓ A canonical stochastic trajectory meta-heuristic based on statistical mechanics. The transition probability from state s to neighbor s' with $\Delta E = f(s') - f(s)$ is governed by the Metropolis criterion:

$$P(s \rightarrow s') = \begin{cases} 1 & \text{if } \Delta E \leq 0 \\ \exp\left(-\frac{\Delta E}{T_k}\right) & \text{if } \Delta E > 0 \end{cases}$$

Temperature T_k is reduced via a geometric cooling schedule $T_{k+1} = \alpha T_k$, $\alpha \in [0.8, 0.99]$, collapsing the algorithm into greedy descent at convergence.

Tabu Search (TS) ✓ A memory-augmented trajectory meta-heuristic. A Tabu List $\mathcal{T}_L$ of recent move attributes (e.g., node pairs, edge identities) explicitly prohibits revisiting recently explored configurations for a configurable tenure. At each step, the best non-tabu neighbor is selected — even if it worsens the objective. An Aspiration Criterion overrides the tabu status if a move achieves a new global best.

Reactive Tabu Search (RTS) ✓ Extends TS with dynamic tenure adaptation. A hash map tracks objective-state revisitations; if a configuration recurs, the tenure is expanded ($T \leftarrow T \cdot \gamma$) to force diversification. If no cycles occur over a defined epoch, the tenure is relaxed to permit localized intensification.

Iterated Local Search (ILS) ✓ A canonical trajectory meta-heuristic defined by a four-step Markov chain: **GenerateInitial** $\rightarrow$ **LocalSearch** $\rightarrow$ **Perturbation** $\rightarrow$ **AcceptanceCriterion**. The search escapes local optima s^* via a perturbation function (e.g., double-bridge 4-opt move) to yield s' , then re-applies local search:

$$s'' \leftarrow \text{LocalSearch}(\text{Perturbation}(s^*))$$

Acceptance of s'' is governed by any pluggable acceptance criterion.

Variable Neighborhood Search (VNS) ✓ Formalizes diversification through nested, dynamically expanding neighborhoods $\mathcal{N}_1, \dots, \mathcal{N}_{k_{\max}}$. A stochastic shaking phase ($s' \leftarrow \text{Random}(\mathcal{N}_k(s))$) is followed by deterministic local search ($s'' \leftarrow \text{LocalSearch}(s')$). If s'' improves upon the incumbent, the search recenters on s'' and resets to $k = 1$; otherwise, the radius expands ($k \leftarrow k + 1$) until $k_{\max}$ is reached.

Guided Local Search (GLS) ✓ A penalty-based trajectory meta-heuristic. It monitors expensive, frequently traversed edges in incumbent solutions and penalizes them to force diversification. The augmented cost $h(s)$ evaluated by local search is:

$$h(s) = f(s) + \lambda \sum_{i=1}^M p_i I_i(s), \quad \text{util}(i) = \frac{c_i}{1 + p_i}$$

Upon reaching a local optimum, the feature with maximum utility is penalized, redirecting subsequent local search to unexplored topological regions.

Guided Fast Local Search (GFLS) $\times$ An acceleration architecture that integrates Fast Local Search (FLS) within the Guided Local Search (GLS) meta-heuristic. Instead of exhaustively evaluating the $\mathcal{O}(N^2)$ or $\mathcal{O}(N^3)$ neighborhood at each iteration, GFLS maintains an active neighborhood bit-mask. Nodes are evaluated only if their activation bit is set to True. When the GLS controller penalizes an edge, the activation bits of the incident nodes are flipped back to True, explicitly directing the subsequent local descent (e.g., 2-opt, Relocate) strictly toward the penalized features. If a node’s full neighborhood is evaluated without yielding an improving move, it is deactivated. This dramatically reduces computational overhead by pruning dormant search regions while preserving the exact trajectory escape properties of GLS.

Adaptive Large Neighborhood Search (ALNS) ✓ A destroy-and-repair meta-heuristic where operator selection is governed by a multi-armed bandit. Operator weights w_j are updated via exponential smoothing with reward c (scaled by improvement quality) and smoothing factor ρ :

$$p_j = \frac{w_j}{\sum_{k \in \Omega} w_k}, \quad w_{j,t+1} = (1 - \rho)w_{j,t} + \rho c$$

The bandit continuously biases selection toward historically successful destroy-repair pairs, with acceptance governed by a Simulated Annealing criterion.

Large Neighborhood Search (LNS) ✓ The non-adaptive predecessor of ALNS. A single fixed destroy operator ejects a subset of nodes U from the incumbent; a fixed repair operator reinserts them. Move acceptance is typically governed by a pluggable criterion (SA, TA, GD). LNS is distinguished from local search by its large move size — the destroyed neighborhood may encompass 10–40% of the solution — bridging topologically distant regions unreachable by edge-exchange operators.

Slack Induction by String Removal (SISR) ✓ A domain-specific meta-heuristic for heavily constrained routing (time windows, capacity). SISR removes sequences (strings) of spatially proximate vertices, probabilistically inducing capacity "slack" into the remaining routes. Let $\bar{c}$ be the average removal string length; the slack enables tight recompilation of the destroyed route segments without capacity infeasibility during repair.

GENIUS ✓ A deterministic local search utilizing Generalized Insertion (GENI) and Unstringing operators. Instead of standard adjacency insertions, GENIUS evaluates non-adjacent vertex placements by executing localized segment reversals. The net spatial cost of inserting vertex v using reference nodes i, j, k, l is:

$$\Delta C = d(i, v) + d(v, j) - d(i, k) - d(l, j) + d(k, l)$$

Fast Iterative Localized Optimization (FILO) ✓ A domain-specific routing heuristic based on spatial granularity. Instead of evaluating the full objective for every perturbation, FILO restricts edge-exchange evaluations to a spatially restricted k -NN subgraph, reducing neighborhood evaluation from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log K)$.

5.3.2 Population and Evolutionary Methods

Genetic Algorithm (GA) ✓ A canonical evolutionary meta-heuristic simulating natural selection. A population of routing solutions (chromosomes) is evaluated via a fitness function $f(x)$. Parents are selected via binary tournament or roulette wheel. Specialized crossover operators (Ordered Crossover, Edge Recombination) maintain topological validity. Mutation introduces spontaneous swaps or inversions with probability p_m .

Hybrid Genetic Search (HGS) ✓ A state-of-the-art memetic algorithm (Vidal, 2022) combining a GA with aggressive local search intensification. To preserve diversity in highly constrained routing spaces, HGS employs a bi-criteria biased fitness:

$$BF(p) = fit(p) + \left(1 - \frac{nc_{pop}}{N}\right) \Delta(p)$$

where nc_{pop} counts population clones and $\Delta(p)$ measures normalized Hamming distance to nearest population neighbors. Every crossover offspring immediately undergoes a steepest-descent TSP solver before population re-entry.

HGS with Ruin and Recreate (HGS-RR) ✓ Replaces HGS's static mutation operators with dynamic ALNS-style destroy-and-repair. Large-scale topological defects from crossover are disrupted and repaired using regret-insertion heuristics, rather than weak localized edge swaps.

Memetic Algorithm (MA) ✓ The general hybrid framework combining a population-based evolutionary strategy with trajectory-based local search. Each individual in the population undergoes local search intensification (e.g., VND, 2-opt, SA) after recombination, ensuring offspring improve toward the local basin floor before competing for survival. MAs consistently outperform pure GAs on VRP benchmarks due to the local search gradient. Key specialized variants:

- **MA with Dual Population (MA-DP):** Maintains two interacting sub-populations — one elitist (preserving high-quality solutions) and one diverse (enforcing novelty) — exchanging individuals periodically via a migration policy.
- **MA with Island Model (MA-IM):** Partitions the population into geographically isolated sub-populations (islands) that evolve independently and exchange individuals via migration events at fixed intervals, preventing global premature convergence.
- **MA with Tolerance-Based Selection (MA-TS):** Replaces strict elitist survivor selection with a tolerance-bounded criterion: offspring are retained even if slightly inferior to parents, up to a configurable fitness tolerance ϵ . This mitigates premature convergence without sacrificing selection pressure.

Differential Evolution (DE) ✓ A population-based continuous meta-heuristic. For target vector $x_{i,t}$, a mutant vector is constructed via:

$$v_{i,t} = x_{r1,t} + F \cdot (x_{r2,t} - x_{r3,t})$$

A trial vector $u_{i,t}$ is formed via binomial crossover with probability CR ; it replaces the target if $f(u_{i,t}) \leq f(x_{i,t})$. For discrete routing, positions are decoded via random-key encoding.

Quantum Differential Evolution (QDE) ✓ Extends DE by incorporating quantum superposition principles into the population update. Instead of real-valued mutation vectors, individuals are represented as quantum probability amplitudes that collapse to discrete states at evaluation. The quantum rotation gate updates each amplitude toward the global best with a rotation angle derived from the fitness landscape, enabling faster convergence on multi-modal routing landscapes.

Evolution Strategy — $(\mu + \lambda)$ -**ES** ✓ Maintains a parent population of size μ ; generates λ offspring via mutation (e.g., Gaussian perturbation). The next generation selects the best μ from the combined parent-offspring pool, enforcing strict elitism:

$$P_{t+1} \leftarrow \text{SelectBest}(\mu, P_t \cup \text{Offspring}_t)$$

Evolution Strategy — (μ, κ, λ) -**ES** ✓ Mitigates immortal local optima by bounding individual lifespan to κ generations. Any individual exceeding its lifespan is strictly discarded, forcing the search to continuously explore novel genetic material even at temporary objective cost.

Evolution Strategy — (μ, λ) -**ES** ✓ A non-elitist strategy where the next generation is selected exclusively from the λ offspring (requiring $\lambda \geq \mu$), discarding all parents. This naturally prevents population stagnation at local optima:

$$P_{t+1} \leftarrow \text{SelectBest}(\mu, \text{Offspring}_t)$$

5.3.3 Swarm Intelligence-Based Methods

K-Sparse Ant Colony Optimization (KS-ACO) ✓ An algorithmic regularization of canonical ACO for dense graphs. The spatial graph is pre-processed into a sparse k -NN graph, restricting ant transition evaluations to the K nearest geometric neighbors:

$$P_{ij}^k = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in N_i^k} [\tau_{il}]^\alpha [\eta_{il}]^\beta}$$

This reduces per-iteration complexity from $\mathcal{O}(V^2)$ to $\mathcal{O}(VK)$ without sacrificing solution quality on clustered routing instances.

Particle Swarm Optimization (PSO) ✓ A continuous swarm meta-heuristic. Particles track personal best P_{best} and global swarm best G_{best} . Velocity and position update as:

$$v_i^{t+1} = wv_i^t + c_1r_1(P_{\text{best},i} - x_i^t) + c_2r_2(G_{\text{best}} - x_i^t), \quad x_i^{t+1} = x_i^t + v_i^{t+1}$$

For discrete routing, positions are decoded via random-key permutation encoding.

PSO Memetic Algorithm (PSOMA) ✓ Hybridizes PSO swarm mechanics with trajectory local search. Following velocity-position updates, discrete tour representations undergo periodic local search refinement (2-opt, swap). The locally optimized tour is inverse-mapped to update particle coordinates, transforming PSO into a macro-level diversification controller over intensified solutions.

PSO with Distance-Based Algorithm (PSODA) ✓ Augments standard PSO with a distance-based repulsion mechanism that explicitly prevents particle clustering in the search space. Particles that fall within a minimum pairwise distance threshold d_{min} of each other trigger a repulsion update, diversifying the swarm before local search collapse.

Artificial Bee Colony (ABC) ✓ Mimics honey bee foraging. Employed bees explore neighborhoods via:

$$v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj}), \quad \phi_{ij} \in [-1, 1]$$

Onlooker bees select food sources proportional to fitness $P_i = F_i / \sum_n F_n$. Sources not improving after a **limit** of cycles are abandoned; scout bees reinitialize them randomly, preventing permanent stagnation.

Firefly Algorithm (FA) ✓ A swarm meta-heuristic based on bioluminescent attraction. Firefly i moves toward brighter firefly j with attractiveness decaying with distance r_{ij} :

$$\beta(r) = \beta_0 e^{-\gamma r_{ij}^2}, \quad x_i^{t+1} = x_i^t + \beta_0 e^{-\gamma r_{ij}^2} (x_j^t - x_i^t) + \alpha(\text{rand} - 0.5)$$

Harmony Search (HS) ✓ Inspired by musical improvisation. A Harmony Memory (HM) stores candidate solutions. New solutions are composed component-by-component: with probability HMCR a component is sampled from HM (and perturbed with probability PAR by $bw \cdot \epsilon$); otherwise it is generated randomly. HM is updated if the new harmony improves upon its worst member.

Sine Cosine Algorithm (SCA) ✓ A trigonometric swarm meta-heuristic. Agent position updates toward destination P_i via switching sine/cosine functions:

$$x_i^{t+1} = \begin{cases} x_i^t + r_1 \sin(r_2) |r_3 P_i^t - x_i^t| & r_4 < 0.5 \\ x_i^t + r_1 \cos(r_2) |r_3 P_i^t - x_i^t| & r_4 \geq 0.5 \end{cases}$$

where r_1 decays over time to tighten the search radius.

League Championship Algorithm (LCA) ✓ Models a sports league competition. Each solution is a team competing in a round-robin schedule. Teams (solutions) update their strategies by learning from their wins and losses against opponents, with the update rule analogous to a social learning model: each team moves toward the strategy of the winning team and away from the strategy of the losing team, weighted by a randomized learning rate.

Volleyball Premier League (VPL) and Variants ✓ A population meta-heuristic modelling volleyball team competition and league structure. Teams are partitioned into leagues; intra-league competition drives local intensification while inter-league promotion/relegation drives diversification. Extended variants include:

Hybrid VPL (HVPL) ✓ Integrates a secondary local search phase within each competition round, hybridizing swarm competition with trajectory descent.

Soccer League Competition (SLC) ✓ A swarm meta-heuristic modelling professional football league dynamics. Players (solutions) belong to teams, which compete in weekly matches. Player positions are updated based on the scoring outcomes: players on winning teams reinforce their current strategy (exploitation), while players on losing teams undergo formation changes (exploration). The league table ranking drives selection pressure, with top-ranked teams' strategies acting as attractor basins for the rest of the population.

5.4 Hyper-Heuristics

Hyper-heuristics operate at a higher abstraction level than classical meta-heuristics. Rather than directly searching the solution space, they search the *space of heuristics*, managing and sequencing portfolios of Low-Level Heuristics (LLHs) to adapt to the problem's topological landscape during execution.

5.4.1 Genetic Programming Hyper-Heuristic (GP-HH) ✓

A generative hyper-heuristic that evolves a mathematical scoring function for constructive routing. Represented as a GP expression tree, the function evaluates candidate insertions using tactical features (`node_profit`, `distance_to_route`, `insertion_cost`, `remaining_capacity`). A K-NN candidate list reduces time complexity from $\mathcal{O}(N^2R)$ to $\mathcal{O}(NKR)$. Fitness is evaluated via average normalized profit across structurally distinct training environments, penalized by a parsimony coefficient. Evolutionary operators: 80% subtree crossover, 20% point mutation or Ephemeral Random Constant perturbation, governed by Koza-style depth limits.

5.4.2 Guided Indicators Hyper-Heuristic (GIHH) ✓

An adaptive selection hyper-heuristic utilizing episodic weight updates via roulette wheel selection over a Multi-Objective Pareto Archive. Operator selection is guided by two indicators: **ScoreA** (Quality Reward) — frequency of offspring entering the non-dominated front — and **ScoreB** (Directional Reward) — objective-space bias toward revenue maximization vs. cost minimization. Directional updates trigger only when an operator's bias opposes the current dominant deviation, enforcing Pareto equilibrium. Structurally identical clones are rejected from the archive to prevent explosion.

5.4.3 Hidden Markov Model + Great Deluge Hyper-Heuristic (HMM-GD-HH) ✓

An online-learning selection hyper-heuristic modelled as an Input-Output Hidden Markov Model (IOHMM). Hidden states (improving, stagnating, escaping) expand dynamically at rate $\mathcal{O}(\sqrt{\log t})$.

LLHs are taken as inputs u_t ; multi-objective solution changes map to a discrete observation alphabet o_t .

1. **Action Selection.** Selects the LLH $\hat{u}$ maximizing the balance between expected normalized profit and transition entropy, annealed by α_t : $\hat{u} = \arg\max_u \left[\bar{P}(u) - \alpha_t \sum_{s'} P(s'|u) \ln(P(s'|u) + \epsilon) \right]$
2. **Belief Update.** The hidden state belief vector is updated via stochastic online EM (Forward algorithm approximation):

$$b_t(j) \propto B_{u_t, j, o_t} \sum_i b_{t-1}(i) A_{u_t, i, j}$$

3. **Acceptance.** Candidate solutions pass a Great Deluge criterion with monotonically rising water level $W_{t+1} = W_t + \lambda |f(s^*)|$.

5.4.4 Reinforcement Learning — Great Deluge Hyper-Heuristic (RL-GD-HH) ✓

An adaptive parameter controller integrating a utility-based RL mechanism with Great Deluge acceptance (Ozcan et al., 2010). Each heuristic i maintains a scalar utility $u_i \in [0, U_{\max}]$. The highest-utility operator is always selected (random tie-breaking). On strict improvement: $u_i \leftarrow \min(U_{\max}, u_i + r)$. On neutral or worsening moves, punishment is applied via one of three variants: subtractive (RL1: $u \leftarrow \max(lb, u - p)$), divisional (RL2: $u \leftarrow \lfloor u/2 \rfloor$), or root (RL3: $u \leftarrow \lfloor \sqrt{u} \rfloor$). Candidate solutions are gated by a linearly updating Great Deluge water level.

5.4.5 Sequence-Based Selection Hyper-Heuristic (SS-HH) ✓

An online-learning hyper-heuristic (Kheiri, 2014) that constructs and evaluates variable-length LLH *sequences* before application. Two dynamically updated matrices drive selection: **TMatrix** — the success probability of executing LLH j immediately after i — and **ASMatrix** — whether to extend the current sequence ($AS = 0$) or terminate and apply it ($AS = 1$). On a new global best, all involved matrix entries receive a proportional reward scaled by the normalized objective improvement Δ_{norm} . Candidate sequences are evaluated through a time-decaying threshold acceptance criterion.

5.4.6 Hyper-Heuristic Ant Colony Optimization (HH-ACO) ✓

A swarm-based meta-algorithmic controller operating strictly within the heuristic space. Artificial ants construct fixed-length sequences of local search and ruin-recreate operators. The transition probability from operator i to j follows the ACS pseudo-random proportional rule, balancing exploitation (probability q_0 threshold selecting $\arg \max[\tau_{ij}^\alpha \eta_{ij}^\beta]$) and exploration (roulette wheel). The pheromone matrix τ_{ij} is bounded by MMAS constraints ($\tau_{\min}, \tau_{\max}$) with evaporation rate ρ to reinforce highly synergistic operator sequences.

5.4.7 Hyper-Heuristic with Unstringing/Stringing and Local Search K-opt (HULK) ✓

An advanced domain-specific operator management architecture (Müller & Bonilha, 2022). Governs a tripartite operator pool: Unstringing (generalized destruction), Stringing (greedy/regret reconstruction), and Local Search (intra/inter-route k -opt and swaps). An ϵ -greedy Adaptive Operator Selector tracks each operator’s historical success within a sliding memory window. Selection weights are recalibrated via a dual-decay learning rate:

$$\text{weight} \leftarrow (1 - \text{lr}) \cdot \text{weight} \cdot \text{decay} + \text{lr} \cdot \text{avg_score}$$

Candidate move acceptance is governed by the Boltzmann–Metropolis criterion with the correct sign convention:

$$P = \exp\left(-\frac{\Delta f}{T}\right)$$

as temperature T cools over successive epochs.

Route Construction Algorithms & Solvers

Table 12: Hierarchy of algorithmic frameworks for constructing and optimizing routing solutions.

Framework Level	Key Algorithms	Core Mechanism
Exact Solvers	Branch-and-Price-and-Cut	Unifies column generation (ESPPRC) and cutting planes for mathematically tight LP bounds.
	ILS-BD / LBBD	Benders Decompositions utilizing integer optimality cuts or CP logic for stochastic routing.
Matheuristics	POPMUSIC	Decomposes massive instances via KD-Tree spatial indexing for targeted, localized exact solves.
	Local Branching	Exact solver bounded by Hamming-distance constraints to prune the MILP search tree.
Meta-Heuristics	ALNS / LNS	Orchestrates ruin-and-recreate operators via an adaptive multi-armed bandit.
	Hybrid Genetic Search	Population-based evolutionary search heavily augmented with TSP/local search intensifiers.
Hyper-Heuristics	GP-HH / HMM-GD-HH	Meta-controllers that sequence, evolve, or select low-level heuristics using online learning/Markov models.

Route Construction: Algorithm Selection Guide

Table 13: Recommended construction frameworks based on problem scale and constraints.

Problem Scale / Constraint	Recommended Frameworks
Establishing provable mathematical baselines	Branch-and-Price-and-Cut, Set-Partitioning
Stochastic optimization with recourse	ILS-BD, Progressive Hedging, Exact SDP
Fast generation of high-quality initial states	GRASP, CF-RS (Cluster-First Route-Second)
Massive scale scaling ($N > 1000$)	POPMUSIC, ALNS, Hybrid Genetic Search
Adapting to unknown instance topologies	HMM-GD-HH, SS-HH, HULK

Route Construction: Category Trade-off Summary

Table 14: Trade-offs across route construction algorithm abstraction tiers.

Abstraction Tier		Optimality	Speed	Scalability	Adaptability
Exact & Decomposition		Guaranteed	Very Slow	Low	Low (Model specific)
Matheuristics		Bounded	Slow	Medium	Medium
Trajectory	Meta-	Heuristic	Fast	High	High (Operator driven)
Heuristics					
Population	Meta-	Heuristic	Medium	High	High (Genetic diversity)
Heuristics					
Hyper-Heuristics		Learned	Medium	Very High	Very High (Self-tuning)

6 Acceptance Criteria

In trajectory-based meta-heuristics, the acceptance criterion defines the mathematical rule determining whether the search transitions from a current solution s to a generated candidate solution s' . Assuming an objective function $f(\cdot)$ subject to minimization, the criteria below are organized along two axes: the information they use to reach a decision (cost-only vs. structural/historical vs. population-level), and the determinism of the outcome (fully deterministic vs. stochastic). Let $\Delta f = f(s') - f(s)$.

6.1 Greedy & Elitist Criteria

These criteria accept moves based solely on immediate, deterministic cost comparisons with no tolerance for objective deterioration. They form the bedrock of descent-based local search and serve as the acceptance core for tabu search.

6.1.1 Only Improving (OI) ✓

The strictest greedy, elitist acceptance rule. The search trajectory is monotonically non-increasing, accepting a candidate only if it yields a strict objective reduction:

$$f(s') < f(s) \quad (\Delta f < 0)$$

Used as the inner loop of pure hill-climbing and steepest-descent local search.

6.1.2 Improving and Equal (IE) ✓

A weakly elitist strategy that accepts solutions of equal or superior quality:

$$f(s') \leq f(s) \quad (\Delta f \leq 0)$$

Unlike OI, IE admits random walks across neutral plateaus (fitness-flat regions), mitigating premature stagnation in combinatorial landscapes with large degenerate regions.

6.1.3 Tabu Criterion (TC) ×

The operational acceptance core of Tabu Search. A candidate move ($s \rightarrow s'$) is **accepted** if and only if the move is not classified as tabu — i.e., the move attribute does not appear in the tabu list $\mathcal{T}$:

$$\text{Accept}(s') \iff \text{attr}(s \rightarrow s') \notin \mathcal{T}$$

The tabu list $\mathcal{T}$ holds recently applied move attributes (e.g., node identities, edge pairs) for a configurable tenure $|\mathcal{T}|$ to explicitly prohibit revisiting recently explored configurations. The Aspiration Criterion (below) acts as its override mechanism. Together, TC and AC define the full tabu search acceptance rule.

6.1.4 Aspiration Criterion (AC) ✓

An override mechanism for Tabu Search that supersedes the TC rejection when a tabu-classified candidate achieves a strict new global best:

$$\text{Accept}(s') \iff f(s') < f(s^*)$$

Even if $\text{attr}(s \rightarrow s') \in \mathcal{T}$, the move is admitted because its quality strictly surpasses any previously discovered solution, preventing the tabu mechanism from blocking globally superior solutions.

6.1.5 All Moves Accepted (AMA) ✓

An extreme, non-selective rule that unconditionally accepts every generated candidate:

$$P(\text{accept } s') = 1$$

Equivalent to a pure random walk. Used as a naive diversification baseline or as a stress test for operator libraries.

6.2 Thermodynamic & Stochastic Criteria

These criteria introduce controlled stochasticity into the acceptance decision, permitting occasional uphill moves with a probability that depends on the magnitude of deterioration and a tunable control parameter. This allows the search to escape local optima basins while still being biased toward improvement.

6.2.1 Boltzmann–Metropolis Criterion (BMC) ✓

The canonical thermodynamic acceptance rule, derived from statistical mechanics (Kirkpatrick et al., 1983). Improving moves are accepted deterministically; deteriorating moves are accepted with a probability governed by the Boltzmann distribution:

$$P(\text{accept } s') = \begin{cases} 1 & \text{if } \Delta f \leq 0 \\ \exp\left(-\frac{\Delta f}{T}\right) & \text{if } \Delta f > 0 \end{cases}$$

The artificial temperature $T > 0$ is reduced over time via a cooling schedule. Common schedules include:

- **Geometric:** $T_{k+1} = \alpha T_k$, $\alpha \in (0.9, 0.999)$ — the standard default
- **Linear:** $T_k = T_0 - k \cdot \delta$ — uniform cooling
- **Hyperbolic:** $T_k = T_0 / (1 + k)$ — slower initial cooling
- **Logarithmic:** $T_k = T_0 / \ln(1 + k)$ — theoretically convergent but practically slow

The cooling schedule is the primary tuning lever of Simulated Annealing solvers.

6.2.2 Cauchy (Lorentz) Simulated Annealing ×

An alternative to BMC that replaces the Boltzmann (Gaussian-generating) distribution with a Cauchy (Lorentz) distribution, yielding heavier probability tails for accepting large deteriorations:

$$P(\text{accept } s') = \frac{1}{1 + (\Delta f/T)^2}$$

Because the Cauchy distribution has infinite variance, this criterion accepts large worsening moves far more frequently than BMC at the same temperature, dramatically increasing global exploration. It is particularly effective in landscapes with deep, widely-separated basins separated by high barriers.

6.2.3 Adaptive Boltzmann–Metropolis Criterion (ABMC) ✓

An advanced BMC variant where the temperature schedule is not predetermined but updated dynamically based on real-time search feedback — such as the rolling variance of accepted Δf values or a target acceptance rate $\hat{p}$:

$$T_{k+1} = T_k \cdot \exp\left(\frac{\hat{p} - p_k}{\beta}\right)$$

where p_k is the empirical acceptance rate over a recent window and β is a sensitivity constant. This removes the need to pre-tune the cooling schedule by closing the feedback loop between the temperature and the landscape geometry.

6.2.4 Generalized Tsallis Simulated Annealing (GTSA) ✓

Grounded in non-extensive Tsallis statistical mechanics, GTSA replaces the exponential Boltzmann distribution with a q -exponential, enabling a heavier-tailed and tunable probability distribution for accepting deteriorating moves:

$$P(\text{accept } s') = \left[1 - (1 - q) \frac{\Delta f}{T}\right]^{\frac{1}{1-q}}$$

The entropic parameter $q \in \mathbb{R}$ governs tail weight: $q \rightarrow 1$ recovers the standard Boltzmann distribution; $q > 1$ yields super-Gaussian tails (more permissive of large deteriorations); $q < 1$ yields sub-Gaussian tails (stricter). Tsallis SA exhibits faster convergence than standard SA on certain multi-modal landscapes where frequent long-range jumps are beneficial.

6.2.5 Monte Carlo (MC) ✓

A static-temperature Metropolis rule: either fixes T at a constant throughout the search (rather than cooling it), or applies a fixed parameterized probability p to accept any worsening move regardless of Δf magnitude. Used primarily as a controlled-temperature baseline or in outer-loop exploration phases where cooling is undesirable.

6.2.6 Exponential Monte Carlo with Counter (EMCC) ✓

A lightweight alternative to full SA that modulates the acceptance probability of deteriorating moves via an exponentially decaying iteration counter c , eliminating the need for a formal temperature schedule:

$$P(\text{accept } s') = \exp\left(-\frac{\Delta f}{c}\right)$$

The counter c decreases monotonically with search iterations, mechanically tightening the acceptance threshold over time. Computationally cheaper than SA because it avoids temperature-reheating or schedule tuning.

6.2.7 Probabilistic Transition (PT) ✓

A state-dependent stochastic acceptance rule that defines acceptance probability via a Markov transition function $P(s \rightarrow s')$, explicitly conditioned on the current search state and its position in the trajectory history. Unlike BMC — which depends only on Δf and T — PT may incorporate features such as solution age, visit frequency, or distance from the global best to bias the Markov chain toward historically productive regions of the landscape.

6.2.8 Fitness Proportional (FP) ✓

A stochastic rule where the acceptance probability is proportional to the relative fitness of s' against the incumbent s :

$$P(\text{accept } s') = \frac{F(s')}{F(s) + F(s')}$$

where $F(\cdot)$ is a non-negative fitness transformation of f (e.g., $F(s) = 1/f(s)$ for minimization). Widely used in evolutionary local search and population selection mechanisms; it naturally biases acceptance toward superior candidates while admitting inferior ones at lower but non-zero probability.

6.3 Deterministic Bounded-Deterioration Criteria

These criteria deterministically admit or reject a candidate based on an explicit, bounded tolerance for objective worsening — no probabilistic sampling is involved. They provide the predictability of greedy descent with a controlled margin of flexibility to escape shallow local optima.

6.3.1 Threshold Accepting (TA) ✓

A deterministic analogue to Simulated Annealing (Dueck & Scheuer, 1990). A candidate is accepted if the objective degradation remains within a monotonically decreasing threshold τ :

$$f(s') - f(s) \leq \tau \quad \Leftrightarrow \quad \Delta f \leq \tau$$

τ begins at a large initial value (permitting broad exploration) and is reduced toward zero as the search progresses, tightening the criterion to pure hill-climbing at convergence. Because acceptance is deterministic — no random sampling — two runs with identical inputs are fully reproducible.

6.3.2 Record-to-Record Travel (RRT) ✓

A deterministic, deviation-bounded rule relative to the global best rather than the current incumbent (Dueck, 1993). A candidate is accepted if its cost is within a fixed scalar deviation δ of the best-known solution s^* :

$$f(s') \leq f(s^*) + \delta$$

RRT exhibits a natural self-regulating property: as s^* improves, the absolute acceptance ceiling $f(s^*) + \delta$ automatically tightens, eliminating the need to schedule the deviation parameter. δ is typically held constant or reduced with a simple linear schedule.

6.3.3 Great Deluge (GD) ✓

A deterministic acceptance rule that uses an absolute cost boundary — the "water level" W — rather than a relative threshold (Dueck, 1993). A candidate is accepted if and only if its cost remains below the current water level:

$$f(s') \leq W$$

W is initialized above the starting solution cost (for maximization problems, below it) and decreases monotonically at a fixed decay rate ΔW per iteration, forcing convergence by progressively narrowing the acceptance window.

6.3.4 Non-Linear Great Deluge (NLGD) ✓

Extends GD by decaying the water level W non-linearly — exponentially or logarithmically — rather than by a fixed scalar per step:

$$W_{k+1} = W_k \cdot e^{-\lambda}$$

The non-linear schedule enables rapid initial exploration (high W , broad acceptance) that asymptotically tightens as the search converges, closely mirroring the effective behaviour of geometric-cooling SA but with a fully deterministic acceptance test.

6.3.5 Demon Algorithm (DA) ✓

A bounded-deterioration criterion that maintains an internal energy reserve $D \geq 0$ (the "demon"). Worsening moves are accepted if — and only if — the demon possesses sufficient energy to absorb the cost increase:

$$\Delta f \leq D \Rightarrow \text{Accept}; \quad D \leftarrow D - \Delta f$$

Improving moves replenish the demon's energy: $D \leftarrow D + |\Delta f|$. The demon therefore acts as a self-regulating, move-history-aware acceptor: it is strict when the search has been improving (low D) and permissive after a streak of deterioration has depleted it. Unlike SA, the demon's energy is not externally scheduled — it emerges organically from the search trajectory.

6.4 Structural & Distance-Based Criteria

These criteria base the acceptance decision not solely on cost but on the structural distance or novelty of the candidate relative to the incumbent or best-known solution. They are particularly valuable in meta-heuristics operating on combinatorial landscapes where cost similarity does not imply structural similarity.

6.4.1 Skewed Variable Neighborhood Search (SVNS) ✓

An acceptance criterion that permits worsening moves if the candidate is sufficiently far — in structural solution space — from the current incumbent (Hansen et al., 2006). This counteracts the tendency of pure cost-based criteria to cluster the trajectory in a narrow structural neighbourhood:

$$f(s') - \alpha \cdot \rho(s, s') < f(s)$$

where $\rho(s, s')$ is a structural distance metric over the solution space (e.g., number of differing edges, Hamming distance over assignment vectors) and $\alpha > 0$ is an asymmetry parameter scaling the distance bonus. Candidates that are both geometrically remote and only modestly worse than s are admitted; candidates that are spatially adjacent and costly are rejected.

6.4.2 Population-Distance Acceptance (PDA) ×

A criterion for population-based frameworks (evolutionary algorithms, memetic algorithms, island models) that conditions acceptance on the structural novelty of the candidate relative to the current population $\mathcal{P}$, rather than solely against the incumbent:

$$\text{Accept}(s') \iff f(s') < \bar{f}(\mathcal{P}) \vee \rho(s', \mathcal{P}) > \rho_{\min}$$

where $\bar{f}(\mathcal{P})$ is the population mean cost and $\rho(s', \mathcal{P}) = \min_{p \in \mathcal{P}} \rho(s', p)$ is the minimum structural distance from s' to any existing population member. A candidate that is below average cost is accepted unconditionally; a candidate that is above average but structurally unique (distant from all population members) is also admitted to maintain diversity. This prevents premature convergence without requiring a global temperature parameter.

6.5 Memory & History-Based Criteria

These criteria evaluate acceptance against historical search data — a sliding window of past costs, a step counter, or a dynamically adjusted threshold based on acceptance/rejection streaks. They adapt the tolerance automatically to the recent search trajectory without requiring an externally scheduled parameter.

6.5.1 Late Acceptance Hill-Climbing (LAHC) ✓

A memory-based criterion that compares the candidate against the cost of the solution visited exactly L steps ago, rather than the current incumbent (Burke & Bykov, 2017):

$$f(s') \leq f(s_{i-L})$$

A circular cost-history array of length L is maintained. When the search is in a flat or slowly improving region, $f(s_{i-L}) \approx f(s)$ and the criterion behaves like IE; when the search has been declining rapidly, the historical cost acts as a looser threshold that permits temporary uphill steps. The array length L is the sole tuning parameter, and its value implicitly governs both the acceptance temperature and the effective search radius.

6.5.2 Step Counting Hill Climbing (SCHC) ✓

A discrete memory criterion that replaces the continuous history array of LAHC with a single static cost bound B that is refreshed every L accepted steps:

$$f(s') \leq B; \quad B \leftarrow f(s) \text{ every } L \text{ steps}$$

During the L -step window, B is held fixed and acts as a flat acceptance ceiling. Upon expiry, B is updated to the current incumbent, resetting the cycle. SCHC is strictly simpler than LAHC (one variable vs. an array of length L) while achieving comparable performance on many VRP benchmarks, making it attractive for embedded or memory-constrained implementations.

6.5.3 Old Bachelor Acceptance (OBA) ✓

A non-monotone, adaptive threshold that dynamically adjusts based on the recent acceptance or rejection trajectory (Hu et al., 1995). If a move is accepted, the threshold is **tightened** (mimicking high selectivity after a recent success); if a streak of consecutive rejections occurs, the threshold is **progressively relaxed** to force trajectory escape:

$$\tau_{k+1} = \begin{cases} \tau_k - \delta_{\text{tight}} & \text{if last move accepted} \\ \tau_k + \delta_{\text{relax}} & \text{if last move rejected} \end{cases}$$

The acceptance rule is then $f(s') - f(s) \leq \tau_k$. OBA is self-regulating: a long stagnation streak will automatically lower the bar, while a run of improvements will raise it, avoiding the manual specification of cooling schedules. The two adjustment deltas δ_{tight} and δ_{relax} are its primary tuning parameters.

6.6 Multi-Objective & Ensemble Criteria

These criteria operate on vector-valued objective functions or aggregate decisions from multiple independent criteria. They are the acceptance backbone of multi-objective meta-heuristics, Pareto-archive managers, and ensemble solvers.

6.6.1 Pareto Dominance (PD) ✓

The fundamental strict acceptance criterion for multi-objective landscapes. A candidate s' replaces the incumbent s if and only if it strictly dominates it across all objectives $i = 1, \dots, m$:

$$f_i(s') \leq f_i(s) \forall i \quad \wedge \quad f_j(s') < f_j(s) \text{ for at least one } j$$

In practice, Pareto-based meta-heuristics maintain a non-dominated archive and accept a candidate if it is non-dominated with respect to the archive, adding it to the archive and removing any solutions it dominates.

6.6.2 ϵ -Dominance (ϵ -Dom) ✓

A relaxed Pareto criterion that accelerates convergence by admitting candidates whose objective improvements are mathematically significant but numerically negligible. A candidate s' ϵ -dominates s if:

$$(1 - \epsilon)f_i(s') \leq f_i(s) \quad \forall i$$

This prevents infinitesimally marginal improvements from stalling the search or cluttering the Pareto archive. The parameter $\epsilon > 0$ sets the granularity of the dominance check; larger ϵ coarsens the Pareto front, accelerating convergence at the cost of archive precision.

6.6.3 Stochastic Pareto Acceptance (SPA) ×

A probabilistic relaxation of strict Pareto dominance designed for noisy or uncertain multi-objective landscapes. Rather than requiring strict dominance, SPA accepts s' with a probability inversely proportional to the degree of inferiority:

$$P(\text{accept } s') = \exp\left(-\frac{\max_i \max(0, f_i(s') - f_i(s))}{T}\right)$$

The parameter T controls the stringency; as $T \rightarrow 0$, SPA recovers deterministic Pareto dominance. SPA is used in multi-objective simulated annealing variants where evaluations are stochastic and exact dominance comparisons are unreliable.

6.6.4 Tournament Acceptance ✓

The candidate s' is evaluated against a randomly sampled subset $\mathcal{C} \subset \mathcal{P}$ of κ competitors drawn from the current population or archive. It is accepted into the active trajectory if it strictly outperforms a required fraction of those competitors:

$$|\{c \in \mathcal{C} : f(s') < f(c)\}| \geq \kappa_{\min}$$

Tournament acceptance is the standard mechanism in genetic algorithms and memetic algorithms for survivor selection, balancing selection pressure (via κ) and population diversity (via random competitor sampling).

6.6.5 Ensemble Move Acceptance (EMA) ✓

A meta-decision architecture that evaluates a candidate move through a portfolio of heterogeneous criteria concurrently — for example, BMC, GD, and IE simultaneously. The final acceptance decision is aggregated via configurable logical ensemble rules:

- **G-AND (Strict Consensus):** Accept only if all criteria agree — conservative, high-precision
- **G-OR (Authority Rule):** Accept if any single criterion votes positively — aggressive, high-diversity
- **G-VOT (Majority Vote):** Accept if a strict majority of criteria are satisfied — balanced
- **G-PVO (Probabilistic Vote):** Accept with probability proportional to the fraction of satisfied criteria — smooth interpolation between G-OR and G-AND

EMA avoids the brittleness of any single criterion while leveraging the complementary strengths of a diverse portfolio. The ensemble composition itself can be subject to adaptive reweighting (e.g., via Thompson Sampling over the criteria) to favour the best-performing criterion on the current instance.

Acceptance Criteria in Trajectory-Based Meta-heuristics

Table 15: Mathematical rules governing trajectory transitions and objective deterioration tolerance.

Category	Criterion	Type	Acceptance Rule / Feature
Elitist	Only Improving (OI)	Deterministic	$\Delta f < 0$ (Strict greedy descent).
	Improving & Equal (IE)	Deterministic	$\Delta f \leq 0$ (Permits random walks across neutral plateaus).
Thermodynamic	Boltzmann (BMC)	Stochastic	Accepts worsening moves with probability $\exp(-\Delta f/T)$.
	Cauchy SA	Stochastic	Utilizes heavier-tailed probability for escaping deep, wide basins.
Bounded	Great Deluge (GD)	Deterministic	Cost must remain below a monotonically dropping water level W .
	Record-to-Record	Deterministic	Cost must be within margin δ of the global best ($f(s^*) + \delta$).
Memory-Based	Late Acceptance (LAHC)	Deterministic	Compares candidate against the cost from exactly L steps ago.
	Old Bachelor (OBA)	Adaptive	Threshold tightens on success and relaxes on consecutive rejections.
Multi-Objective	Pareto Dominance (PD)	Deterministic	Candidate must strictly dominate the incumbent across all objectives.

Acceptance Criteria: Selection Guide

Table 16: Recommended acceptance criteria based on search landscape and algorithm goals.

Search Landscape / Goal	Recommended Criteria
Strict descent to nearest local minimum	Only Improving (OI), Improving & Equal (IE)
Escaping deep, wide attractor basins	Cauchy (Lorentz) SA, Generalized Tsallis SA (GTSA)
Avoiding complex parameter/temperature tuning	Late Acceptance (LAHC), Great Deluge (GD)
Self-regulating bounds based on history	Old Bachelor Acceptance (OBA), Demon Algorithm
Multi-objective Pareto optimization	Pareto Dominance, ϵ -Dominance
Population/Evolutionary survivor selection	Tournament Acceptance, Fitness Proportional

Acceptance Criteria: Category Trade-off Summary

Table 17: Trade-offs across acceptance criteria categories.

Category	Exploration	Exploitation	Tuning Difficulty	Primary Case	Use
Greedy & Elitist	Low	Very High	None	VND, pure hill-climbing	
Thermodynamic	High	Medium	High	Simulated Annealing	
Bounded-Deterioration	Medium	High	Low-Medium	Threshold Accepting, RRT	
Memory & History	Medium	High	Very Low	Adaptive local search, LAHC	
Multi-Objective	N/A	N/A	Low	Pareto front generation	

7 Route Improvement and Refinement Strategies

Route improvement algorithms operate on pre-existing, structurally feasible routing configurations to systematically reduce operational cost or maximize net profit. The taxonomy below progresses from fast, localized topological operators to exact mathematical reformulations and machine-learning-augmented orchestrators. Algorithms within each category share the same fundamental mode of interaction with the solution — what they differ in is neighborhood scope, acceptance mechanism, and the mathematical guarantee (if any) attached to the output.

7.1 Dynamic Node Insertion

These algorithms augment an existing feasible routing topology by inserting currently unassigned or highly profitable nodes, dynamically modifying route assignments without destroying the overall structural skeleton.

7.1.1 Cheapest Insertion Augmentation ✓

A greedy constructive heuristic. For every unassigned node k and valid insertion position between adjacent nodes (i, j) in active routes, the marginal detour penalty is computed:

$$\Delta C_{k,ij} = d_{i,k} + d_{k,j} - d_{i,j}$$

In profit-aware environments, the effective gain is $\Delta G_{k,ij} = r_k - \Delta C_{k,ij}$. The node-position pair yielding the maximum economic or spatial gain is committed greedily, and the process iterates until no profitable insertion remains. Time complexity per iteration: $\mathcal{O}(|U| \cdot |V_{\text{active}}|)$.

7.1.2 Regret- k Insertion Augmentation ✓

An advanced constructive heuristic that mitigates the spatial myopia of pure greedy insertion. For each unassigned node i , insertion costs are evaluated across its k best viable positions. The regret value — the opportunity cost of not placing i in its optimal position — is:

$$\text{Regret}_i = \sum_{j=2}^k (\Delta C_{i,j} - \Delta C_{i,1})$$

Nodes with maximum regret are prioritized: the algorithm first commits nodes that would suffer the greatest cost increase if deferred. This prevents the greedy order from systematically under-serving spatially isolated nodes with limited insertion opportunities.

7.1.3 Profitable Detour Augmentation ✓

A spatially-constrained heuristic for the Vehicle Routing Problem with Profits (VRPP). For an active edge (u, v) and an unassigned node b , the spatial detour is computed as $\Delta d = d_{u,b} + d_{b,v} - d_{u,v}$. A node is inserted only if the detour satisfies a proportional threshold ($\Delta d \leq \epsilon \cdot d_{u,v}$) and the net economic gain is positive:

$$\Delta \text{Profit} = r_b - C \cdot \Delta d > 0$$

The threshold ϵ controls the geometric reach of insertion; lower values enforce tighter route adherence while higher values permit broader detours for high-revenue nodes.

7.1.4 Path-Induced Synergistic Pickup ✓

An opportunistic zero-cost insertion filter exploiting fleet physical trajectories. If an unassigned node physically lies along the geometric shortest-path sequence between consecutive scheduled stops (u, v) — that is, $d_{u,b} + d_{b,v} = d_{u,v}$ — and sufficient vehicle capacity exists, the node is assimilated at zero marginal routing cost. Particularly effective in dense urban topologies where many nodes lie geometrically co-linear.

7.2 Intra-Route Intensification

These solvers treat each individual vehicle route as an isolated Traveling Salesperson Problem, optimizing the visit sequence strictly within a single route without altering inter-route node assignments. They are used as a polish step following construction or inter-route restructuring.

7.2.1 Steepest 2-Opt Refinement ✓

A deterministic intensification operator. It exhaustively evaluates the complete 2-opt neighborhood — all possible edge-crossing removals — for each route independently. It commits to the single permutation yielding the maximum cost reduction and repeats until no improving 2-opt move exists, guaranteeing convergence to a strict 2-opt local minimum. Time complexity per pass: $\mathcal{O}(|R|^2)$ per route.

7.2.2 Or-Opt Sequence Relocation ✓

A targeted intra-route improvement operator that extracts contiguous node chains of length $L \in \{1, 2, 3\}$ and reinserts them at every other position within the same route. It is strictly more powerful than 2-opt for sequences of length ≥ 2 because it permits non-reversing relocations that 2-opt cannot achieve. The steepest variant evaluates the full neighborhood before committing; the first-improvement variant commits upon the first discovered improvement.

7.2.3 Dynamic Programming (DP) Exact Reoptimization ✓

An exact intra-route solver applying the Held-Karp dynamic programming algorithm. It guarantees the globally optimal node sequence for any route assigned to a single vehicle. Executing in $\mathcal{O}(|R|^2 \cdot 2^{|R|})$ time and space, it is gated by a hard cardinality limit (typically $|R| \leq 20$) and used selectively on short routes or as a post-processing polish on the final solution.

7.2.4 Fast TSP Refinement ✓

A high-speed, scalable local search heuristic. Individual routes are isolated, stripped of depot connections, and delegated to a highly optimized C++ TSP backend for intra-sequence optimization. The backend applies a combination of 2-opt, 3-opt, or Or-opt moves using efficient data structures (neighbor lists, don't-look bits) to guarantee local optimality before the route is reintegrated into the multi-route solution.

7.2.5 Lin-Kernighan (LK) Refinement ✓

The foundational variable-depth intra-route heuristic. Rather than evaluating edge exchanges of a fixed cardinality k (as in 2-opt or 3-opt), it dynamically constructs sequential k -opt moves. It iteratively removes and adds edges to the tour, maintaining a running positive gain, and unbounds the depth k until no further sequential improvement can be found. This allows the search trajectory to bypass shallow local minima that trap fixed-depth operators.

7.2.6 Lin-Kernighan-Helsgaun (LKH) Refinement ✓

The premier heuristic for the TSP sub-problem. LKH dynamically constructs variable-depth k -opt sequential exchanges, beginning from a promising 2-opt or 3-opt improving move and recursively extending the exchange chain until no further gain is achievable. The Helsgaun extension restricts the candidate set at each step using α -nearness bounds derived from the minimum 1-tree, containing the otherwise exponential search space while enabling 5-opt and partition moves that standard local search cannot reach. Produces near-optimal solutions on instances with thousands of nodes.

7.2.7 Lin-Kernighan-Helsgaun 2 (LKH-2) Refinement ✓

An advanced extension of the LKH architecture. While the original LKH relies strictly on symmetric α -nearness 1-trees, LKH-2 generalizes the variable-depth search to accommodate asymmetric edge costs (ATSP), time windows, and multiple depots via sophisticated problem transformations and tour partitioning techniques. It efficiently maps complex constrained VRP sub-problems into symmetric TSP variants that its highly optimized 5-opt and general k -opt engine can resolve, extending LKH's near-optimal precision to heavily constrained problem classes.

7.3 Inter-Route Topological Operators

These algorithms operate simultaneously across multiple vehicle routes, performing structural recombinations that balance load, reallocate nodes between vehicles, and optimize spatial boundaries. They are the primary mechanism for escaping the intra-route local optima left by single-route intensifiers.

7.3.1 2-opt* (Inter-Route Edge Exchange) ✓

The canonical inter-route extension of 2-opt (Potvin & Rousseau, 1995). Two routes R_a and R_b are selected. An edge (u, u') in R_a and an edge (v, v') in R_b are simultaneously removed and replaced with cross-route links (u, v) and (u', v') , effectively swapping the route tails:

$$\Delta C = d_{u,v} + d_{u',v'} - d_{u,u'} - d_{v,v'}$$

Unlike intra-route 2-opt, this move does not reverse any sub-tour — both new segments inherit the orientation of their respective parent routes. Capacity feasibility is checked on both recombined routes before commitment. 2-opt* is often the first inter-route operator applied because its $\mathcal{O}(|V|^2)$ neighborhood is fast to evaluate and frequently yields large improvements.

7.3.2 Node Relocation (Inter-Route Relocate) ✓

Removes a single node u from its current route R_a and inserts it at its cheapest feasible position in a different route $R_b \neq R_a$. The marginal cost delta is:

$$\Delta C = (d_{p_u, n_u} - d_{p_u, u} - d_{u, n_u}) + (d_{v, u} + d_{u, n_v} - d_{v, n_v})$$

where p_u, n_u are the predecessor and successor of u in R_a , and v, n_v are the insertion position in R_b . Node Relocation is the single-node analogue of Or-opt applied inter-route, and is typically the highest-frequency improving move in CVRP local search. A full pass evaluating all (u, v) pairs runs in $\mathcal{O}(|V|^2)$.

7.3.3 Steepest Node Exchange ✓

An exhaustive steepest-descent operator computing the exact objective delta for all possible pairwise node swaps $(i \leftrightarrow j)$ across both intra- and inter-route neighborhoods. It irrevocably commits to the single optimal transposition per iteration. The inter-route capacity constraint $\sum_{k \in R_a} q_k - q_i + q_j \leq Q$ is checked before each candidate swap is scored.

7.3.4 Or-Opt Refinement (Steepest and Iterative) ✓

A generalization of intra-route relocation applied across the full multi-route solution. Contiguous chains of length $L \in \{1, 2, 3\}$ are extracted from one route and reinserted into any feasible position across all routes — including the source route — providing both intra- and inter-route chain relocation in a single operator. The steepest variant maps the entire neighborhood exhaustively before committing; the iterative first-improvement variant exploits the first discovered positive delta for faster convergence.

7.3.5 Cross-Exchange Local Search ✓

An advanced inter-route operator generalizing 2-opt* to contiguous segment swaps. It exchanges a segment of up to L consecutive nodes from route R_a with a segment from route R_b , simultaneously severing and reconnecting four distinct edges. This enables massive structural recombinations — effectively performing a constrained subtour exchange — that 2-opt* (which swaps only tails) cannot produce.

7.3.6 Swap-Star ✓

A refined inter-route swap that addresses the geometric suboptimality of blind position-preserving swaps. Rather than naively swapping nodes $u \in R_a$ and $v \in R_b$ in-place, Swap-Star evaluates the globally optimal insertion position for u within R_b and for v within R_a , selecting the pair that maximizes the net combined cost improvement. It produces strictly better solutions than standard pairwise swap at the cost of an $\mathcal{O}(|R|)$ inner loop per candidate pair.

7.3.7 Classical Local Search ✓

A deterministic trajectory-based refinement wrapping a suite of inter-route operators (3-opt, 4-opt, Or-opt, Swap-Star). It evaluates localized edge permutations systematically, driving the multi-route solution to a strict local minimum via iterative or steepest-descent improvement. Serves as the intensification backbone within VNS, ALNS, and memetic algorithm frameworks.

7.4 Trajectory Meta-Heuristics

These frameworks introduce controlled stochasticity or structured memory into the local search loop to accept objective deterioration deliberately, navigating the trajectory out of deep local optima basins that pure greedy descent cannot escape.

7.4.1 Variable Neighborhood Descent (VND) ✓

A deterministic meta-heuristic that systematically sequences through a predefined ordered set of neighborhood structures $\mathcal{N}_1, \mathcal{N}_2, \dots, \mathcal{N}_k$. Starting from $\mathcal{N}_1$, local search is applied until a local minimum is reached. The search then transitions to $\mathcal{N}_2$ to attempt improvement in the larger or differently structured neighborhood. If an improvement is found, the method resets to $\mathcal{N}_1$; if $\mathcal{N}_k$ is exhausted without improvement, the current solution is declared the VND local optimum:

$$s^* \leftarrow \text{LocalSearch}_{\mathcal{N}_l}(s); \quad l \leftarrow \begin{cases} 1 & \text{if } f(s^*) < f(s) \\ l + 1 & \text{otherwise} \end{cases}$$

VND is both an effective standalone improver and the standard intensification engine within Variable Neighborhood Search (VNS). A typical VRP VND sequence applies: relocate $\rightarrow$ 2-opt* $\rightarrow$ swap $\rightarrow$ Or-opt $\rightarrow$ cross-exchange.

7.4.2 Tabu Search (TS) Refinement ×

A memory-augmented trajectory meta-heuristic that escapes local optima by explicitly prohibiting recently applied moves for a configurable tenure $|\mathcal{T}|$. At each iteration, the best non-tabu neighbor is selected — even if it worsens the objective — and committed. The tabu list $\mathcal{T}$ is maintained as a circular queue of recently applied move attributes (e.g., node pairs, edge identities):

$$s \leftarrow \arg \min_{s' \in \mathcal{N}(s) \setminus \mathcal{T}} f(s')$$

An Aspiration Criterion overrides the tabu status of any move that achieves a strict new global best $f(s') < f(s^*)$. Reactive Tabu Search adapts the tenure $|\mathcal{T}|$ dynamically: it increases tenure upon detecting cycling (repeated solution visits) and decreases it during stagnation. TS consistently reaches high-quality VRP solutions because its prohibition of revisits systematically diversifies the trajectory beyond the immediate local minimum.

7.4.3 Iterated Local Search (ILS) ✓

A meta-heuristic that alternates between a perturbation phase — which escapes a local optimum by disrupting the solution beyond the reach of the incumbent local search — and an intensification phase — which re-optimizes from the perturbed state. The standard ILS loop is:

$$s_0 \rightarrow \text{LocalSearch}(s_0) \rightarrow \text{Perturb}(s^*) \rightarrow \text{LocalSearch}(s') \rightarrow \text{Accept?}(s', s^*)$$

The perturbation strength is the critical tuning parameter: too weak and the restart falls back to the same basin; too strong and it behaves as a random restart. A Double-Bridge 4-opt move is the canonical VRP perturbation because it creates a topological change that no 2-opt, 3-opt, or Or-opt operator can undo. The acceptance criterion (comparing s' against s^*) may be OI, IE, or any stochastic criterion from the Acceptance Criteria taxonomy.

7.4.4 Simulated Annealing (SA) Refinement ✓

A stochastic trajectory meta-heuristic using thermodynamic perturbations. At each iteration, a neighbor s' is generated via a randomly selected topological operator (2-opt, Or-opt, relocate). Acceptance is governed by the Boltzmann–Metropolis criterion, permitting controlled objective deterioration as an artificial temperature T cools:

$$P(\text{accept } s') = \begin{cases} 1 & \text{if } \Delta f \leq 0 \\ \exp\left(-\frac{\Delta f}{T}\right) & \text{if } \Delta f > 0 \end{cases}$$

The geometric cooling schedule $T_{k+1} = \alpha T_k$ is standard; reheat schedules (periodically raising T) can be added to re-diversify upon stagnation. SA’s primary advantage over TS is parameter simplicity — a single temperature governs all acceptance — at the cost of less directed diversification.

7.4.5 Guided Local Search (GLS) ✓

A penalty-based meta-heuristic that escapes local optima by augmenting the objective function with learned edge penalties. It monitors which edges appear frequently in incumbent solutions that are locally but not globally optimal, assigning a penalty p_{ij} to each. The modified objective penalizes over-exploited edges:

$$U(i, j) = \frac{c_{ij}}{1 + p_{ij}}$$

After each local search convergence, the penalties of the most profitable (most-used but costly) edges are incremented, redirecting the subsequent local search away from previously explored topological regions. GLS is particularly effective in symmetric VRP instances where suboptimal solutions share a small number of highly favoured but costly edges.

7.4.6 Randomized Local Search (RLS) ✓

A stochastic search that bypasses exhaustive steepest descent by randomly sampling an operator and an application point at each iteration. The sampling distribution over operators (Or-opt, cross-exchange, 2-opt*) may be uniform or weighted by historical improvement rates. RLS introduces sufficient stochasticity to escape shallow local optima while remaining significantly faster than full neighborhood evaluation — making it the preferred inner loop for high-iteration-count meta-heuristics operating under strict time limits.

7.5 Large Neighborhood Search (Destroy and Repair)

These algorithms escape local optima by aggressively dismantling a large portion of the current routing topology and intelligently reconstructing it, bridging the trajectory to topologically distant regions of the solution space that local operators cannot reach in a single step.

7.5.1 Ruin and Recreate (LNS) ✓

The foundational destroy-and-repair meta-heuristic. At each iteration, a ruin operator ejects a subset U of nodes from the current solution s (via random, worst, Shaw, or spatial removal), and a recreate operator reinserts them into the remaining partial solution s' (via greedy, regret, or GENI insertion). The candidate s'' is then evaluated against s using a pluggable acceptance criterion — decoupling the exploration amplitude (controlled by the ruin operator) from the acceptance tolerance (controlled by the criterion). This decoupling is LNS’s primary architectural advantage over local search: the move scale is not bounded by neighborhood size.

7.5.2 Adaptive Large Neighborhood Search (ALNS) ✓

A highly adaptive LNS variant where operator selection is governed dynamically by a Multi-Armed Bandit with Thompson Sampling. The bandit maintains a Bayesian Beta posterior $\text{Beta}(\alpha_i, \beta_i)$ of success for each ruin–recreate operator pair, sampling a success probability $\theta_i \sim \text{Beta}(\alpha_i, \beta_i)$ at each iteration and selecting the operator pair with the highest sampled value. Posterior parameters are updated multiplicatively after each application based on the quality of the resulting solution. ALNS outperforms static LNS on heterogeneous instances where no single operator is uniformly dominant — a common condition in multi-period and profit-maximizing VRP variants.

7.6 Exact Solvers

These algorithms evaluate restricted or localized state spaces using rigorous mathematical bounding, guaranteeing absolute optimality for the sub-problems presented to them. Applied to sub-problems rather than the full instance, they act as powerful polish steps that heuristic operators cannot replicate.

7.6.1 Set-Partitioning Polish ✓

An exact mathematical refinement operating on a pre-supplied pool of high-quality routes. It formulates the optimal route combination as a Set-Partitioning MILP:

$$\min \sum_r c_r x_r \quad \text{s.t.} \quad \sum_r a_{ir} x_r = 1 \quad \forall i \in V, \quad x_r \in \{0, 1\}$$

solved to global optimality by a commercial MILP solver (e.g., Gurobi, CPLEX). Particularly effective as a final-stage combinator after ALNS or LNS has generated a diverse route pool, because the set-partitioning problem over a fixed pool is far smaller than the full VRP.

7.6.2 Branch-and-Price (B&P) Refinement ✓

An exact column-generation solver applied as a localized improver. The routing state is formulated as a Set Partitioning Master Problem (RMP). The pricing sub-problem — an Exact Resource-Constrained Shortest Path Problem (ESPPRC) solved with ng -route relaxation and Lagrangian bounding — iteratively generates improving route columns with negative reduced cost:

$$\bar{c}_r = c_r - \sum_{i \in V} \pi_i a_{ir} - \mu < 0$$

Integrality is enforced via Ryan-Foster branching. Applied to a restricted sub-graph (e.g., the routes and nodes involved in the most recent LNS destruction), B&P refinement achieves provable optimality over the affected sub-problem without the intractability of global B&P.

7.7 Matheuristics

Hybrid architectures that explicitly combine the trajectory speed and exploration logic of heuristic search with the mathematical bounding power of MILP formulations, inheriting the best properties of both paradigms.

7.7.1 Set-Partitioning Pool Construction ✓

A matheuristic that automatically builds a massive pool of candidate routes through diverse generation strategies — LNS perturbations, Held-Karp sub-sequence optimization, and mandatory singleton routes. After canonical deduplication, a restricted Set-Partitioning MILP is solved to global optimality over the heuristically generated pool. The quality of the final solution depends directly on the diversity of the pool: a heterogeneous pool from multiple solvers and perturbation seeds consistently outperforms a pool from a single trajectory.

7.7.2 Fix-and-Optimize Matheuristic ✓

A decomposition-based exact strategy that permanently locks a subset of high-quality routes as binary constants, then passes the nodes belonging to the remaining "free" routes to an MILP sub-solver for exact re-optimization:

$$\min_{x \in \{0,1\}^{|F|}} f(x, \bar{x}) \quad \text{s.t.} \quad x_r = \bar{x}_r \quad \forall r \in \text{Fixed}, \quad \text{capacity, covering constraints}$$

The fixed/free partition is selected based on route quality metrics (cost per demand, geographic compactness) and rotated across iterations, allowing the MILP to exactly re-optimize different sub-problems from a globally promising context.

7.7.3 MIP Large Neighborhood Search (Exact Ruin and Recreate) ✓

A matheuristic variant of LNS where the repair phase delegates reconstruction to an exact MILP sub-solver rather than a greedy heuristic. A spatial or randomized ruin operator destroys a portion of the solution; the fragmented nodes are then re-inserted into the remaining locked routes by a sub-MIP that is solved to proven optimality:

$$\min_y \sum_{i \in U} \sum_{r \in \mathcal{R}_{\text{free}}} c_{ir} y_{ir} \quad \text{s.t.} \quad \sum_r y_{ir} = 1 \quad \forall i \in U, \quad \text{capacity on } \mathcal{R}_{\text{free}}$$

This bridges rapid combinatorial exploration (the LNS outer loop) with mathematical exactness (the sub-MIP repair), consistently achieving tighter solutions than heuristic recreate at moderate computational overhead.

7.8 Neural Algorithms

These approaches integrate trained deep learning architectures directly into the improvement or operator-selection process, learning from the topology of routing instances rather than applying generic mathematical rules.

7.8.1 Learned Route Improver (Neural Operator) ✓

A GNN-augmented operator that bypasses the $\mathcal{O}(n^k)$ complexity of exhaustive k -opt evaluation. Multi-layer MLP node and edge encoders embed the current routing state into a latent representation. A neural "move head" directly predicts the expected objective improvement $\hat{\Delta}f$ for each candidate topological exchange, executing only the highest-scoring moves without enumerating the full neighborhood. The GNN is trained offline on solved instances and generalizes to new topologies of the same structural class.

7.8.2 Neural Operator Selector (Neural Orchestrator) ✓

A deep neural policy that governs the high-level operator selection loop rather than executing moves itself. At each iteration, it extracts a state vector encoding the optimality gap, search progress, stagnation count, and recent operator performance history, and produces a probability distribution over the available operator suite via a policy-gradient-trained network. Unlike ALNS's Thompson Sampling bandit — which tracks only aggregate historical success — the neural orchestrator conditions its selection on the current search geometry, learning operator-to-landscape mappings that static bandit mechanisms cannot represent.

7.9 Meta-Algorithmic Orchestrators

Advanced management systems that sequence, govern, or dynamically weight the execution of entire improvement algorithms rather than individual low-level operators. They operate at the strategy level, treating complete algorithms as atomic actions.

7.9.1 Multi-Phase Composition ✓

A meta-algorithmic architecture that pipelines multiple distinct refinement strategies into a sequential execution graph. Complex life-cycles are constructed explicitly — for example: greedy augmentation $\rightarrow$ inter-route SA local search $\rightarrow$ exact LKH polish $\rightarrow$ set-partitioning combination — while tracking granular performance metrics (improvement per second, accepted-move rates, operator utilization) across each discrete transition. Phase boundaries are triggered by configurable conditions such as iteration limits, stagnation thresholds, or improvement-rate drops.

7.9.2 Adaptive Ensemble Route Improver (Meta-ALNS) ✓

A sophisticated orchestrator managing a portfolio of complete high-level route improvement algorithms (LKH, SA, Ruin-Recreate, Fix-and-Optimize) as atomic bandit arms. An Exponential Moving Average (EMA) of historical objective improvements tracks the recent performance of each algorithm:

$$\text{EMA}_i \leftarrow (1 - \lambda) \cdot \text{EMA}_i + \lambda \cdot \Delta f_i$$

Selection is driven by Roulette Wheel sampling over the EMA-normalized weights, continuously re-weighting the portfolio toward the currently most effective strategy. Unlike the low-level ALNS bandit — which operates over destroy/repair operator pairs — Meta-ALNS operates over full algorithmic strategies, enabling coarser but more powerful macro-level adaptation to the current phase of the search.

Route Improvement and Refinement Strategies

Table 18: Algorithms designed to refine and improve pre-existing feasible routing topologies.

Scope	Key Algorithms	Refinement Action
Node Insertion	Cheapest / Regret- k	Greedily or preemptively inserts unassigned nodes to balance marginal cost versus opportunity cost.
Intra-Route	Steepest 2-Opt / DP	Optimizes the sequence within an isolated route (e.g., removing edge crosses or exact Held-Karp DP).
Inter-Route	Cross-Exchange / Swap-Star	Swaps segments or nodes between different vehicle routes to balance load and spatial boundaries.
Trajectory Search	VND / ILS / SA	Chained neighborhood explorations using deterministic descent or stochastic perturbations to escape local minima.
Matheuristic Polish	Set-Partitioning Polish	Solves an exact Set-Partitioning MILP over a heuristically generated pool of high-quality routes.
Neural/Orchestrators	Meta-ALNS / Neural Operator	Manages portfolios of algorithms or uses GNNs to predict high-yielding k -opt edge exchanges.

Route Improvement: Strategy Selection Guide

Table 19: Recommended improvement strategies based on refinement goals.

Refinement Goal	Recommended Strategies
Intra-route sequence optimization (TSP)	LKH Refinement, Steepest 2-Opt, Held-Karp DP
Load balancing across vehicle boundaries	Cross-Exchange, Swap-Star, Inter-Route Relocate
Macroscopic topological escapes	Ruin & Recreate (LNS), Adaptive LNS
Aggregating historically proven routes	Set-Partitioning Polish, Fix-and-Optimize
Bypassing slow exhaustive neighborhood scans	Learned Route Improver (Neural Operator)

Route Improvement: Category Trade-off Summary

Table 20: Trade-offs across route improvement operator categories.

Category	Disruption	Cost per Move	Scope	Solver	De- pendency
Constructive Augmenta- tion	Low	Low	Unassigned $\rightarrow$ Route	Pure	Heuris- tic
Intra-Route Operators	Low	Low-Medium	Single Route	Pure	Heuris- tic
Inter-Route Operators	Medium	Medium-High	Route to Route	Pure	Heuris- tic
Large Neigh- borhood Search	High	High	Macroscopic	Pure	Heuris- tic
Exact Matheuristic Polish	Very Low	Very High	Route Pool	MILP	Solver Required
Neural Or- chestrators	Adaptive	High (Train), Low (Run)	System-wide	GPU	/ Deep Learning